

Metamodel-Guided Model Generation with Layered Constraints

Tong Ma^{1,2}, Hui Lai¹, Hui Wang³, Zhenhu Tian³, Chaochao Li^{1*}, Fengjie Xu^{2*},
Ling Fang^{2*}

¹University of Science and Technology of China, Hefei, China.

²Hefei Institutes of Physical Science, Chinese Academy of Sciences, Hefei, China.

³Anhui University, Hefei, China.

*Corresponding author(s). E-mail(s): fangl@hfcas.ac;

Contributing authors: matong@mail.ustc.edu.cn;

Abstract

Large language models (LLMs) enable natural-language interaction in engineering modeling, but their outputs may violate structural constraints, domain rules, or task requirements. We propose a model generation method based on domain metamodels that combines generation-time constraints with post-generation validation. The method extracts and transforms metamodel information and uses metamodel terminology to guide the extraction of structured constraints from natural-language specifications. It automatically links these constraints to metamodel elements and records their sources in an Integrated Constraint Model (ICM). During task execution, applicable constraints are selected according to the requirements and existing model context. The generation-time constraint layer (L1) restricts candidate structures, values, and references. The post-generation validation layer (L2) checks the model and its serialized artifacts against domain constraints and original task requirements. Validation feedback guides repairs within the permitted scope, followed by renewed acceptance checks. Validation uses existing domain tools and checkers written by humans with LLM assistance. Experiments cover AUTOSAR, railway models, and structured decision-making in private international law. For AUTOSAR, we conducted three runs for each of 20 requirements. All 60 end-to-end runs passed acceptance checks within the specified task scope, and all 255 resulting ARXML files passed XSD validation. A separate local AUTOSAR experiment recorded constraint interventions during decoding. The results show that generation-time constraints and post-generation checks can jointly support model construction. Domain rules and original task requirements provide the basis for detecting violations, guiding repair, and determining acceptance.

Keywords: Model-driven engineering; Large language models; Metamodels; Layered constraints; Integrated Constraint Model

1 Introduction

Model-driven engineering (MDE) treats models as primary artifacts in software and systems development. Explicit modeling languages, model transformations, and analysis tools support design, validation, and implementation. Domain-specific modeling languages express problems in terms of domain concepts.

Their metamodels define the abstract syntax of models, including element types, attributes, inheritance, containment, references, and relationship multiplicities. Well-formedness constraints specify additional conditions that valid models must satisfy. Model transformations connect different levels of abstraction and representations and support the generation of code and other artifacts from models [1]. These mechanisms

define the artifacts on which automated operations act and the criteria against which they are checked.

Large language models (LLMs) allow users to specify modeling requirements in natural language and obtain suggestions for model elements, relationships, and attribute values. Recent studies have applied LLMs to domain modeling, model instance generation, interactive completion, and the transformation of modeling artifacts, combining natural-language interaction with established MDE tools [2]. However, a linguistically plausible response is not necessarily a model that modeling tools can accept. A generator must respect types, required features, nested structures, multiplicities, and references. It must also meet the complex semantic and logical requirements in specifications, as well as task-specific requirements concerning objects, values, and connections. As models become more complex, these requirements increasingly interact during generation.

Structural correctness is itself a substantial part of this challenge. In AUTOSAR, for example, the ARXML representation of a software component model has multiple levels of nesting, different reference types, and element ordering requirements specified by XML Schema. Incorrect containment, missing required elements, or references of the wrong type can prevent an artifact from being used in downstream tools. If JSON is used as an intermediate representation during generation, its correspondence with the domain model must be preserved, and serialization must satisfy the target format requirements. For example, when model features are represented as JSON object properties, the written order of those properties does not enforce the element order required by XSD [3]. The serialization mapping must enforce this order, and validation must check it in the resulting artifact.

Structural requirements, domain rules, and task requirements address different concerns. Consider a task that requires a timing event to trigger an existing runnable entity R every 10 ms. A runnable entity represents software behavior that can be triggered for execution. In the XML representation, omitting the required event name violates a structural requirement. Referencing a runnable entity that does not exist violates reference consistency. A period of 0.02 s fails to satisfy the required 10 ms interval even if the structure and references are correct. Deleting the event during repair may remove a violation, but it also removes the periodic behavior required by the task. Model construction must therefore address **metamodel conformance**, **model consistency** with respect to the specified rules, and **requirements satisfaction** together.

This modeling information is usually distributed across different sources. The metamodel provides structural information such as types and relationships. Natural-language specifications add conditions involving multiple elements, logical dependencies, applicability conditions, and exceptions. The task determines what must be constructed and retained. Generation may also depend on the **existing model context**, which comprises objects in the current model that can be referenced or must be preserved, together with their attributes and relationships. For example, generation should account for the runnable entity R and its containing internal behavior, as well as interfaces and their paths already defined in the component model. Including all this information in a prompt does not make the corresponding constraints executable.

Previous studies have introduced explicit knowledge at different stages of generation. SynchroMesh combines retrieval with constrained semantic decoding to enforce syntax, type, and selected contextual rules as the output is constructed [4]. AbsCon aggregates multiple candidate graphs into a probabilistic partial model and then uses the metamodel and well-formedness constraints to solve for a consistent model [5]. EMF-Kaizen uses the metamodel and the current model to recommend model fragments interactively [6]. These studies show that constraints can guide generation and support candidate checking and selection. For tasks involving both complex structures and domain logic, a key question is how to coordinate two complementary capabilities. Generation-time restrictions should exclude invalid structures and values where possible. Post-generation checks should address requirements that depend on complete objects, relationships across elements, or target artifacts.

We propose a **metamodel-guided method for model generation with layered constraints**. The generation-time constraint layer (L1) and the post-generation validation layer (L2) jointly support model construction subject to structural constraints and domain rules. L1 enforces requirements that can be applied during generation through constrained decoding. L2 uses domain validation tools to check generated models and their serialized artifacts. Task acceptance checks then verify the objects, values, and relationships specified in the original requirements. The two layers are distinguished by when and how constraints can be enforced. Structural and logical requirements are assigned according to their representations and the capabilities of the available backends.

To support this division of responsibilities, the method constructs a reusable **Integrated Constraint Model (ICM)**. It first extracts and transforms information from an existing domain metamodel, preserving element identifiers and their correspondence with the source representation. Metamodel terminology then guides constraint extraction from natural-language specifications. The extraction identifies the elements to which constraints apply, along with their conditions, requirements, and exceptions. The constraints are automatically linked to metamodel elements, and their sources in the specifications are recorded. Metamodel elements provide a shared domain context for interpreting constraints. The ICM organizes structural information and specification requirements from different sources for use in generation, retrieval, and validation.

For each task, the method selects relevant constraints based on the requirements and the existing model. It binds these constraints to the objects, values, and reference targets involved in the task and dynamically assembles generation constraints and the validation configuration. Temporary requirements apply to the current generation and repair process, while reusable domain rules are stored in the ICM. When errors are detected, validation feedback identifies the violated requirements and the affected objects. The repair process uses this feedback to propose changes within the permitted scope. Each repair is checked again against both domain rules and the original task requirements to avoid accepting changes that remove required objects or alter prescribed values.

The method applies to domains that can be described by metamodels, provided that task-relevant constraints can be mapped to generation-time constraints and post-generation validation rules. The domain metamodel is provided as input. Generation mappings and validation tools are incorporated during domain preparation. We use existing domain tools and custom checkers written by humans with LLM assistance.

The paper makes three contributions.

(1) **A layered constraint method that coordinates generation and validation.** The method combines the generation constraints enforced by L1 with the post-generation checks performed by L2 to support model construction subject to structural constraints and domain rules. It retains the original task requirements as acceptance criteria for both generation and repair, providing a process for constrained model construction, validation, and repair.

(2) **ICM construction based on metamodel elements.** Metamodel information is transformed, structured constraints are extracted using metamodel terminology, and the resulting constraints are automatically linked to the elements to which they apply. The ICM records their sources and intended uses, providing reusable constraint information for execution in the two layers.

(3) **An empirical evaluation in three domains.** Experiments with AUTOSAR automotive software models, railway models, and structured decision-making in private international law (PIL) examine model and artifact acceptance, constraint interventions during decoding, repair with validation feedback, and the enforcement of constraints on decisions. The first two domains evaluate the construction of engineering model instances. PIL examines the application of the layered approach to decisions governed by many rules and requiring reliable conclusions.

For AUTOSAR, we conducted three runs for each of 20 requirements. All 60 end-to-end runs passed acceptance checks within the specified task scope, and all 255 resulting ARXML files passed XSD validation. In the controlled fault-injection experiment, all 85 core units and all 15 substitute units were repaired successfully. A separate local AUTOSAR decoding experiment directly observed the role of constraints during generation. The main railway experiment comprised 24 tasks, each run three times. Among the 72 runs, 23 met the strict success criterion after initial generation, compared with 51 after repair with validation feedback. The PIL experiment comprised 60 cases. The full constraint configuration achieved better results in the combined evaluation of structure, rule compliance, and task conclusions. Together, these experiments examine the engineering workflow, execution mechanism, and application of the layered method across domains. The railway tasks that remained unrepaired also inform the analysis of the limits of current generation and repair capabilities.

The remainder of the paper is organized as follows. Section 2 discusses related work. Section 3 presents the layered constraint method and ICM construction. Section 4 reports the AUTOSAR engineering experiment, the local AUTOSAR decoding experiment, the railway experiments, and the PIL experiment in that order. Section 5 concludes the paper and discusses future work. The appendices are organized by experiment and document the sources of the requirements, the setup for extracting constraints from AUTOSAR

specifications, the construction of checkers, and supplementary results. They also include one complete example for each of AUTOSAR, railway modeling, and PIL, from task input to final acceptance.

2 Related work

Related research includes traditional MDE techniques for constructing consistent models from metamodels and constraints, as well as modeling from natural language, structured constraint acquisition, constrained decoding, and repair using feedback.

2.1 Model transformation and consistent model generation

MDE uses modeling languages to define the concepts that can be expressed and the conditions for model validity. Metamodels specify abstract syntax. Well-formedness constraints capture requirements that cannot be fully expressed through structural definitions such as types and relationships, while concrete syntax defines the textual or graphical representation of models. The Object Constraint Language (OCL) expresses invariants, preconditions, and postconditions in a model context and defines their types and evaluation semantics [7]. XML Metadata Interchange (XMI) provides an XML representation for exchanging model objects [8]. Models and their serialized documents therefore require related but distinct checks. Document parseability, metamodel conformance of objects, and compliance with rules involving multiple objects must each be assessed against the appropriate criteria.

Model transformations make correspondences between models or representations explicit. Kahani et al. classify model transformation tools and discuss their support for model-to-model and model-to-text transformations [1]. The two-stage code generation framework proposed by Ma et al. combines XMI structural information with XSD constraints. It first generates static code structures and then adds behavior based on model annotations and associations [9]. These studies illustrate the importance of representation mappings when incorporating industrial modeling languages into a toolchain. Types and structures must be carried across representations, and target artifacts may have organizational requirements that are not directly expressed in the source representation. Reading a metamodel and producing a representation suitable for processing is distinct from creating model

instances, generating code, or producing exchange artifacts, although these operations can be combined in a workflow.

Consistent model generation uses constraints directly during model construction. Semeráth et al. take a metamodel, well-formedness constraints, and a search scope as input. They guide graph solving through the refinement of three-valued partial models, constraint propagation, and incremental checking, and check the original well-formedness constraints after concretization [10]. Their method can start from the most abstract partial model or an initial partial model provided by an engineer, using explicit constraints to narrow the candidate space during search. This provides an MDE basis for combining control during generation with checks after completion. Compared with generation driven by formal constraint solving, generation from natural language must also establish how textual concepts and requirements correspond to model elements that can be manipulated and which constraints apply to the current task.

2.2 LLM-assisted modeling and model instance construction

Research on LLM-assisted modeling covers modeling languages and their metamodels, instances of existing modeling languages, and the completion or modification of current models. Di Rocco et al. review this field in terms of MDE activities, discussing the encoding of modeling artifacts, post-processing of generated results, and evaluation. They also propose a research agenda in which LLMs and MDE support each other [2]. This perspective emphasizes integrating generation into model toolchains, where modeling tasks require more than plain text output.

Combining natural language with a language definition is a common approach to generation. Alaoui Mdaghri et al. generate DSL models from natural-language descriptions and modeling language documentation, validating the results through an iterative process [11]. In a technical report on automotive systems, Petrovic et al. combine Ecore metamodels and requirements in prompts to generate XMI instances. OCL checks determine whether the instances proceed to downstream code generation. The report also explores proposing OCL constraints from reference requirements and metamodels [12]. These studies connect natural-language input, explicit language representations, and subsequent model processing.

Domain configuration generation further illustrates the role of intermediate representations and artifact mappings. El-Gnainy et al. study AI-assisted AUTOSAR configuration, dataset generation, and code production [13]. Samy et al. use a fine-tuned model to generate compact JSON, which is deterministically expanded into ARXML. They evaluate structural, reference, parameter, and semantic conditions for Watchdog Manager configurations [14]. Compact representations reduce the burden of directly generating complex exchange formats, but the expansion rules remain an essential part of the domain implementation. The validity of the intermediate representation and that of the final ARXML artifact must be connected through this mapping.

Interactive assistants integrated into modeling environments use the current model as an evolving context. At the level of abstract syntax, EMF-Kaizen combines the metamodel, current model, user request, and examples to recommend model fragments. It supports metamodel pruning, conversion between JSON and EMF objects, and user selection of fragments for incorporation into the model [6]. Its scope includes both domain models and metamodels, illustrating how assistant capabilities can be reused through shared modeling infrastructure. The reported implementation uses tolerant parsing to handle returned fragments. OCL annotations in the metamodel can be included in prompts, but OCL checks are not performed on the recommended fragments [6]. Interactive recommendation focuses on fragments that a modeler can continue to use, whereas generation for a complete task must also establish whether all required objects and associations are present.

AbsCon combines agreement among candidates with explicit constraints. It first generates multiple candidate graphs from the same text and merges them into a probabilistic partial model. Constraint optimization then selects a concrete model that satisfies the metamodel and well-formedness constraints [5]. Constraints participate in solving for the final model, rather than serving only as a basis for subsequent scoring. In the reported implementation, the metamodel and well-formedness constraints are manually translated into first-order logic [5]. Compared with direct generation or interactive completion, this approach uses information shared across candidates to reduce inconsistencies, showing how model solving and language model generation can complement each other.

2.3 Structured constraint acquisition and traceability

Using natural-language specifications for automation requires identifying the domain concepts, scopes, and logical relationships referred to by rules. Model context can provide types and navigation paths for this interpretation. PathOCL takes a UML class model and natural-language constraints as input, selects relevant classes and model paths to construct prompts, and then generates OCL expressions. The study evaluates compilability and semantic correctness separately. Path-based prompts shorten individual inputs, and attempts with multiple paths increase the proportion of compilable constraints, while gains in semantic correctness are smaller [15]. These results reflect two related problems in constraint acquisition. The text must be associated with an appropriate model context, and its requirements must be correctly expressed as constraints.

Traceability concerns correspondences between artifacts and how those correspondences are used. Winkler and von Pilgrim discuss artifact traceability in requirements engineering and model-driven development, including connections between the two activities [16]. In constraint generation, links to sources and model elements help explain which specification passage supports a rule and which types or relationships it concerns. They also help locate affected content when changes are made. Associating rules with metamodel elements supports reuse across tasks, while binding a rule to specific objects determines the scope of checks for a particular task.

Retrieval-augmented generation combines retrieved content from external documents with text generation by a language model [17]. For engineering specifications, retrieval can supply applicability conditions, exceptions, and background explanations. Structured constraints additionally record explicit links between this information and model elements. The two can be used together. A requirement may provide evidence in a prompt and, where a corresponding mapping exists, support generation control or logical checks. Organizing metamodel information, structuring textual requirements, and implementing executable checks are thus related but distinct activities. Completing text extraction does not by itself produce a checking program.

2.4 Constrained decoding and repair with validation feedback

Constrained decoding filters next-token candidates for validity during generation, allowing encoded structural requirements to influence the output directly. Geng et al. study grammar-constrained decoding for structured tasks and grammars that depend on the input [18]. XGrammar improves the efficiency of structured generation through grammar processing and runtime mask computation [19]. XGrammar-2 further supports dynamic structure switching within a request and cache reuse across grammars [20]. These techniques allow structural restrictions that vary by task to be enforced during decoding. They provide a basis for incorporating metamodel types, required features, enumerations, and representable relationship constraints into model instance generation.

Generation-time control can also incorporate semantic information. Synchromesh combines example retrieval with a completion engine to enforce syntax, scope, type, and selected contextual rules on partial outputs [4]. Projectional Decoding maintains a partial graph model as text is generated and uses the metamodel and incremental semantic checks to filter subsequent tokens. The study reports a preliminary evaluation on a program generation task [21]. The distinction between generation-time and post-generation checks therefore does not impose an absolute separation between syntax and semantics. Whether a condition can be checked early depends on the information already established in the partial model, the executable representation of the constraint, and the backend capabilities. Conditions that require a complete model or context across artifacts can be handled by later checks.

Constraint enforcement also affects the generation distribution. Grammar-Aligned Decoding shows that masking invalid tokens and renormalizing at each step is generally not equivalent to sampling from the original model distribution conditioned on the complete sequence satisfying the grammar. It proposes a sampling method with asymptotic convergence properties [22]. This distinguishes output validity from preservation of the sampling distribution. For engineering modeling, constrained decoding can reduce invalid candidates, but assessing whether a candidate satisfies the meaning of a task still requires appropriate evaluation criteria.

Model consistency maintenance provides mechanisms for locating inconsistencies and selecting

repair operations. Marchezan et al. use rule evaluation to locate causes of inconsistency and construct abstract repair trees containing alternatives and operation sequences. Their approach also supports filtering actions by artifact or user ownership [23]. Research on feedback-based LLM revision investigates how feedback can be used to generate new candidates. Self-Refine uses the same model to generate an output, provide feedback, and iteratively revise the result [24]. SpecGen generates JML specifications for Java programs. It first improves candidates using feedback from verification errors, then combines operator mutations with heuristic selection to find verifiable specifications [25]. These studies show that the source of feedback, available modification operations, and candidate acceptance criteria jointly influence repair outcomes.

Existing research provides capabilities for model transformation, constraint solving, constrained decoding, and repair using feedback. This paper organizes these capabilities through execution in L1 and L2. Requirements on structures and values are enforced during generation where executable, complete models and their artifacts are checked after generation, and repaired results are accepted against the original task requirements. ICM construction links metamodel information, structured constraints from specifications, and their sources. Binding these constraints to the objects involved in a task provides a common basis for both layers. The experiments that follow evaluate engineering models and artifacts, interventions during decoding, and constraint enforcement across domains.

3 Metamodel-guided generation with layered constraints

The method uses an **Integrated Constraint Model** (ICM) to organize metamodel information and domain rules. Task-related requirements are implemented through the **generation-time constraint layer** (L1) and the **post-generation validation layer** (L2). L1 restricts the structure and values of candidate content during generation. L2 checks structural conformance, domain consistency, and task requirements on the constructed model and target artifacts, and provides diagnostics for repair. Shared constraint information and task bindings connect the two layers into a complete process from requirements to model generation, validation, and repair.

3.1 Scope and overview

The method applies to domains that can be described by a **domain metamodel**, provided that task-relevant constraints can be mapped to generation-time constraints and post-generation validation rules. The domain metamodel is provided as input. Other inputs include natural-language domain specifications, the current task requirements, and the **existing model context**. This context consists of model elements that exist before generation and may be used by the current task, together with their attributes and relationships. For example, when adding a timing event to a system, the name, type, and reference path of an existing runnable entity form part of the model context needed by the new event.

The metamodel defines structural information, including available types, attributes, inheritance, containment, references, multiplicities, and enumerations. Natural-language specifications add applicability conditions, exceptions, consistency requirements across elements, and complex semantic and logical requirements. The task further specifies the objects to construct, their numbers and values, their connections, and the target output. Even at the structural level, deep nesting, optional branches, repeated elements, and references can interact. Structural correctness is therefore addressed during generation and checked again after model construction and serialization.

JSON Schema uses validation keywords to describe field types, required fields, numbers of items, and allowed values in JSON instances, providing a constraint representation for structured generation [26]. The method assembles these restrictions for each task and uses them during generation through a constrained decoding backend. XGrammar, for example, can compile the corresponding constraints and use them to filter the tokens permitted during generation [19]. JSON Schema is one implementation choice in this method. Domain adaptations can also use other constraint representations supported by the backend, such as grammars.

A **candidate model** is a model instance constructed according to the domain metamodel and awaiting validation and acceptance. It contains concrete objects, attribute values, and relationships between objects. JSON Schema specifies the structures and values that may be generated for the current task. The generated content becomes a candidate model through model construction. A **serialized artifact** is a concrete representation of the candidate model in the target modeling

language or exchange format, such as an XML file. In structured decision applications, the output may instead be a decision record containing facts, applicable rules, and conclusions. Such outputs reuse the layered constraint enforcement process. Their correspondence with domain concepts is established during domain adaptation and described separately from the construction of engineering model instances.

The overall process comprises domain preparation and task execution. Domain preparation extracts information from an existing metamodel, uses metamodel terminology to guide constraint extraction from natural-language specifications, automatically links constraints to metamodel elements, and records their sources to form a reusable ICM. It also provides generation constraint mappings, model construction and serialization rules, and callable validators. Task execution binds the current requirements to concrete objects and values, assembles L1 generation constraints, retrieves relevant rules, and configures L2 checks. The generated content undergoes model construction and serialization before validation and task acceptance. When a problem can be addressed, its diagnostics guide model changes and revalidation. Figure 1 shows this process.

A simplified ARXML fragment illustrates the three kinds of requirements. The task is to configure a timing event that triggers an existing runnable entity R every 10 ms. A **runnable entity** represents software behavior that can be triggered for execution. The model context supplies the path /Component/Behavior/R for R. The following fragment omits the enclosing component and behavior structures, uses a simplified path, and expresses the period in seconds.

```
<TIMING-EVENT>
  <SHORT-NAME>TE_R</SHORT-NAME>
  <START-ON-EVENT-REF DEST="RUNNABLE-ENTITY">/Component/Behavior/R</START-ON-EVENT-REF>
  <PERIOD>0.01</PERIOD>
</TIMING-EVENT>
```

Omitting the required event name is a structural issue. A reference to a nonexistent runnable entity is an issue with model relationships. A period of 0.02 s fails to meet the task requirement of 10 ms even if the structure and references are correct. These requirements do not map rigidly to different stages. A reference target and period that have already been determined can restrict generation in advance, while the results of model construction and serialization still require post-generation checks.

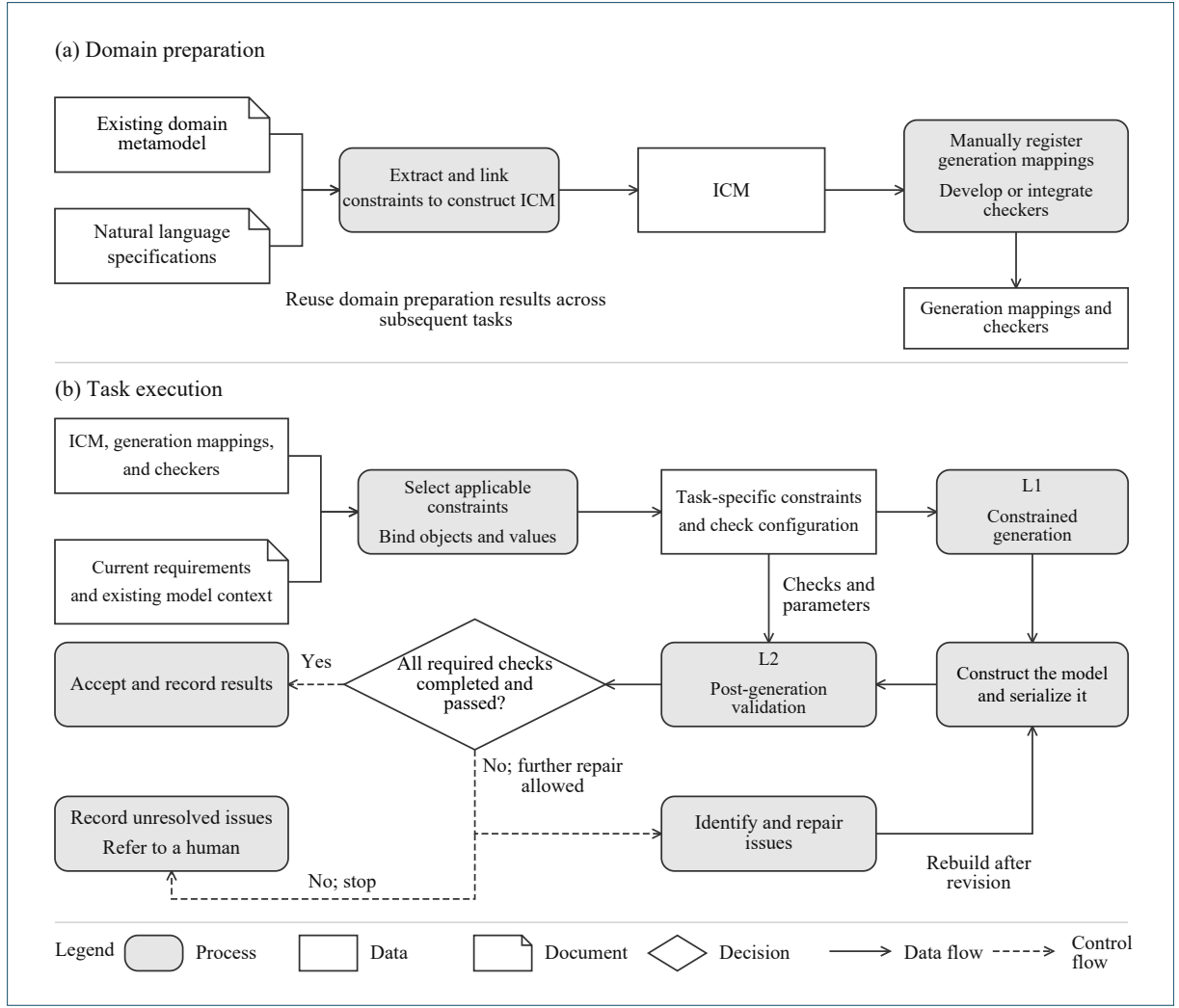


Fig. 1 Domain preparation and execution of an individual task. The ICM, generation mappings, and validators are prepared for reuse across tasks. Task binding determines the content needed for generation, retrieval, and validation. L1 constrains generation, L2 checks the constructed results and guides repair, and task acceptance is determined against the original requirements

The domain integrator provides generation mappings and validators. In this work, custom checkers are written by humans with LLM assistance, while existing tools perform native format checks.

3.2 Extracting and transforming metamodel information

ICM construction begins by transforming an existing metamodel into a queryable intermediate representation. The transformation reads element identifiers, attribute types, inheritance, containment, references, multiplicities, and enumerations, while preserving source element identifiers and versions. Domains with a separate exchange syntax also require mappings

between metamodel elements and constructs in the target language. Examples include the XML attribute or element corresponding to a model attribute and the wrapper nodes corresponding to a containment relationship.

The result can be represented as an object model, relational records, JSON, or a graph. The essential requirement is to preserve the information and source element correspondences needed for extraction, querying, and binding, rather than to adopt a particular storage format. Model-driven language engineering distinguishes abstract syntax, concrete syntax, and the relationships between their evolution [27]. Accordingly,

the metamodel representation describes domain constructs, the serialization mapping defines their target representation, and the generation constraints specify the structures and values allowed for the current task. Each serves a different role in the process.

The transformed types, attributes, and relationships provide standard terminology for text extraction. They also support the selection of types for task objects, the construction of relationships, and the identification of applicable constraints. Constraints are associated with type or relationship definitions in the metamodel. The objects specified by the current task are concrete instances of those types.

3.3 Metamodel-guided constraint extraction and ICM construction

The ICM combines the structural information provided by the metamodel with rules from natural-language specifications. Specification fragments are processed individually. The system adds relevant metamodel types, attributes, relationships, and enumeration terms to the extraction context. It instructs the LLM to extract the elements to which each constraint applies, its applicability conditions, requirements, and exceptions in a common record format, while retaining the specification version and fragment location. Metamodel terminology provides a shared conceptual basis for interpreting the text so that the extracted constraints can be linked directly to domain elements.

The system then links the extracted records automatically to metamodel elements. It resolves target names, checks target types and the ownership of attributes, relationships, or enumeration literals, and establishes references from constraints to the corresponding elements. Records with failed links, ambiguous names, or mismatched members are referred for human handling. The person responsible consults the specification and metamodel to complete or correct the records, after which the links are checked again. Existing structured rules and domain check definitions can also be incorporated into the ICM under the same information requirements, reusing their rule content and sources. Figure 2 shows how the ICM is constructed from natural-language specifications, and Table 1 lists the main information in a constraint record.

We summarize these information relationships by representing the ICM as

$$I = (M, C, A, P) \quad (1)$$

Here, I denotes the integrated constraint model for the target domain. M is the metamodel representation obtained through extraction and transformation, including element identifiers, types, attributes, relationships, multiplicities, and enumerations. C is the persistent set of domain constraint records. It includes constraints extracted from specifications and metamodel structural rules that need to be referenced individually. A contains the associations established between constraint records and metamodel elements. P contains provenance and its links to metamodel elements and constraint records, including the source metamodel or schema files and the versions and specific locations of natural-language specifications. Constraint records retain intended uses, links to existing implementations, and link-checking results.

Figure 3 shows these four kinds of information. Constraints and metamodel elements can have many-to-many associations. A reference consistency rule may involve the source type, the reference relationship, and the target type, while the same type may participate in several rules. Provenance records allow a constraint to be traced back to the original specification and its related types and attributes to their metamodel definitions.

Structured constraints in the ICM have the three uses listed in Table 2. A constraint can serve more than one purpose. Parts that can be mapped to restrictions on structures and values are incorporated into generation constraints. Complete rules and their explanations can be retrieved for inclusion in prompts. Requirements that must be evaluated on a complete model provide a basis for validator development. Intended uses and existing implementations are recorded separately. The ICM thus organizes executable constraints alongside information needed for retrieval and validator development.

This organization preserves the connections between structural information, textual rules, and the information used to enforce them. A requirement can restrict candidate content during generation, support checks on the final relationships after generation, and be traced to the relevant rule and model elements when a failure occurs. The full domain specification need not be reinterpreted for every task.

3.4 Binding task objects and requirements

The persistent ICM describes domain information reused across tasks. The current requirements specify

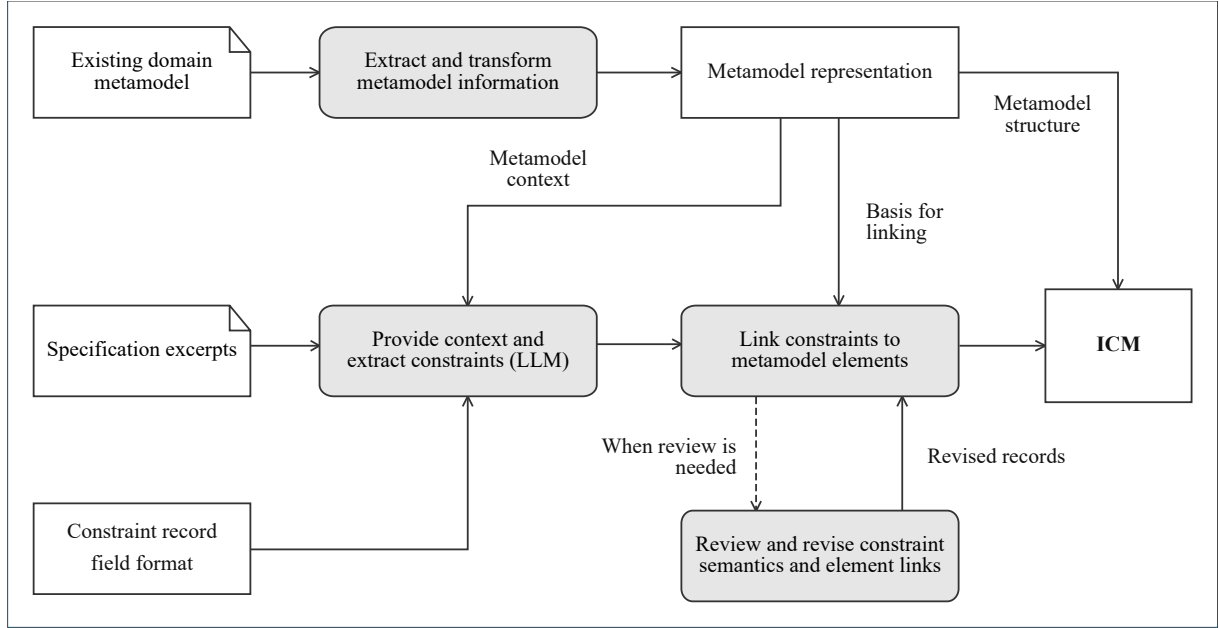


Fig. 2 ICM construction from an existing domain metamodel and natural-language specifications. The metamodel representation supplies terminology for extraction, information for automatic linking, and structural information. Linking issues are referred for human handling. The ICM retains constraint content, element associations, sources, and intended uses

Table 1 Main information in an ICM constraint record

Information	Content	Subsequent use
Identifiers and sources	Rule identifier, specification version, fragment location, and original wording	Trace constraint sources and distinguish versions
Constraint content	Scope, applicability conditions, requirements, and exceptions	Determine when, on which objects, and what to check
Metamodel element associations	Relevant classes, attributes, containment, or references	Retrieve by domain concepts and bind task objects
Uses and implementation links	Intended uses and identifiers of generation mappings or checkers	Connect generation, retrieval, and validation
Link checks	Results and issues from checking target names, types, and member ownership	Locate unmatched or conflicting associations and support correction

Table 2 Three uses of ICM constraints

Use	Application	Example
Generation-time structural constraints	Use existing mappings to restrict types, required items, quantities, enumerations, or reference candidates	Restrict references to targets with matching types in the context
Retrieval for prompting	Retrieve rules and context according to the elements and conditions involved in the task	Provide applicability conditions and specification explanations for references
Basis for validator development	Humans implement or integrate checks from rules and the metamodel, with LLM assistance	Check whether the final reference target exists and has the correct type

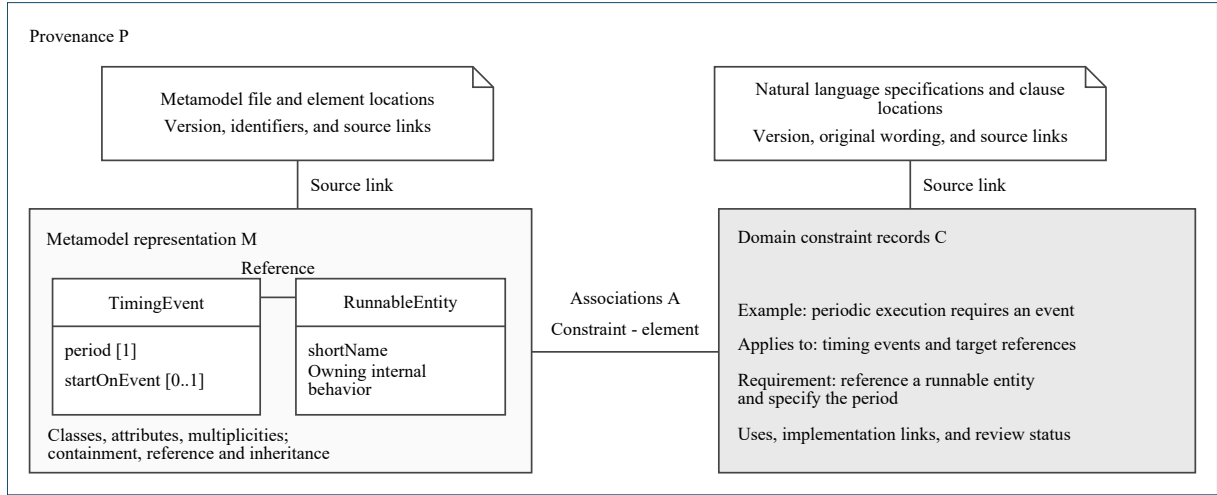


Fig. 3 ICM information and associations. The metamodel representation M provides domain elements and relationships, constraint records C express rule content, associations A link constraints to the elements to which they apply, and provenance P supports tracing back to sources. The three uses of the constraints share this information and do not constitute additional metamodel levels

concrete objects, quantities, values, and connections. Using the existing model context, the system identifies relevant metamodel elements, selects applicable constraints, and binds the requirements to concrete objects in the task. Temporary constraints extracted from natural-language requirements apply only to the current task and its repairs. They are not automatically written back to the persistent ICM.

Depending on domain complexity, task execution can provide information and generate content in multiple stages. Requirements are first analyzed to determine the constituent objects and their dependencies. Relevant metamodel information, specification fragments, and available references are then provided for each part as the model is generated progressively. Designs formed in earlier stages supply structural choices for subsequent constraint assembly, while generated objects that are available for use update the model context. Object identities and values explicitly stated in the requirements remain the basis for the task, and LLM-proposed designs must correspond to them.

The binding results serve three purposes. Requirements with generation mappings enter the L1 generation constraints. Retrieved specification content is added to the prompt as external knowledge [17]. Requirements evaluated on the constructed results are bound to the input objects and parameters of existing checkers. A new check configuration can thus be assembled at runtime without generating new validation code for every task.

In the timing event example, the runnable entity R and the 10 ms period come from the current requirements. Binding determines the target reference and the period value, respectively. The requirement that a reference point to an object of an appropriate type is a reusable rule. Task acceptance checks the target and period against the original requirements. Another runnable entity of the correct type cannot replace the specified R.

3.5 Assembling generation constraints and constrained decoding

3.5.1 Dynamic assembly of generation constraints

L1 assembles the executable parts of the metamodel structure, applicable domain rules, and task bindings into generation constraints for the current task. JSON Schema can express restrictions on field types, required fields, quantities, fixed values, and enumerations [26]. Dynamic assembly uses the metamodel as its structural basis, task choices to determine which parts to expand, and the existing model context to constrain available references. The generation specification therefore varies with the task and the context obtained so far.

Algorithm 1 summarizes the assembly process when JSON Schema is used. Domain adaptation provides type and relationship queries, constraint mappings, target structure templates, and serialization mappings. At runtime, these resources are used to

Algorithm 1 Task-specific JSON Schema assembly

Inputs are the ICM, current task bindings, existing model context, domain structure and serialization mappings, and the constraints supported by the backend.

Outputs are the generation schema for the current call and the correspondences needed for subsequent model assembly.

1. Use task bindings to identify the required types, objects, and dependencies in the metamodel representation.
 2. Expand required structures and task-selected branches, mapping types, required features, and multiplicities to field restrictions.
 3. Use task requirements and the existing model context to bind fixed values, permitted enumeration values, and available reference candidates.
 4. Incorporate domain constraints supported by existing mappings, retaining their correspondence with ICM rules and task requirements.
 5. Check the assembled constraints, adapt them to the decoding backend, and output the generation schema and model assembly correspondences.
-

expand the structure required by the task recursively. Object identities, quantities, and assignments that have already been determined can be encoded as fixed restrictions or assembled by deterministic procedures. For content still to be chosen by the LLM, the assembled constraints retain the permitted values.

Reference candidates are determined jointly by target type, scope, and model context. When the task specifies a target, the reference is bound to that object. When a choice remains, the eligible candidate set is retained. For targets yet to be constructed in the same task, existence, type, and relationship consistency are checked after assembly. Assembly rules handle deterministic content, while requirements that need the complete result are configured for L2.

Generation constraints and the target language do not always have a one-to-one correspondence. For an XML target, XML Schema can require child elements to follow a sequence [3]. When JSON objects carry named fields, their written order does not directly encode this XML ordering requirement. In the corresponding adaptation, the serializer orders elements according to the target schema, and XML Schema validation checks the artifact. This illustrates the division of responsibilities among generation constraints, model construction, and target representation. Other intermediate representations or decoding directly in the target language can use different mappings.

3.5.2 Constrained decoding for model instance generation

In MDE, generated output must be parseable, support the construction of model instances, and be usable by downstream tools. Incorrect nesting, omitted required items, or invalid enumeration values can cause this process to fail before domain validation begins. **Constrained decoding** applies encoded structural restrictions at each generation step. It allows the LLM to choose content within the permitted representation space, providing input that satisfies the generation specification for subsequent model construction [18].

A decoding backend under direct control first converts the generation specification into a supported grammar representation and compiles it using the tokenizer configuration [19] [20]. During generation, the matcher maintains the grammar state from the current output prefix and determines the tokens allowed at the next step. After the LLM produces next-token scores, the backend sets the scores of disallowed tokens to negative infinity and selects the next token from the allowed candidates. The token is appended to the output and the matcher is updated. This process continues until the output is complete.

This control covers punctuation and nesting as well as field names and valid values encoded in the schema. When generating a reference value, the constraints can restrict the output to the candidate paths configured for the task. For a numeric field with a bound value, they can restrict it to the value specified by the task. Tokens do not correspond one-to-one to characters or grammar symbols, so the backend must maintain these permitted choices at the tokenizer level [19]. Figure 4 separates task constraint assembly from this stepwise enforcement process.

Retrieved prompt content and constrained decoding serve complementary roles. The former supplies the meaning of the requirements and relevant domain information, while the latter excludes continuations that violate the structural restrictions being enforced. Constrained decoding changes the output distribution of the original LLM. Stepwise filtering and renormalization generally differ from sampling the original model distribution conditioned on the complete output satisfying the constraints [22]. L1 therefore enforces the generation constraints that have been encoded and are supported by the backend. Domain consistency and requirements satisfaction remain subject to L2 checks.

3.5.3 From generated content to models and serialized artifacts

Once the complete output is returned, the system performs JSON Schema validation against the schema submitted for the call. It checks fields, types, required items, quantities, and encoded values. It then uses the recorded assembly correspondences to add deterministic content, create model objects, organize containment, and establish references. If the generation interface uses a projected intermediate structure, the reconstructed result must also be checked against the original generation constraints after assembly. These checks maintain the connections between the interface response, model assembly, and task constraints.

The domain adapter then converts the candidate model into target artifacts. Serialization handles the wrapper structure, namespaces, reference representation, and element order required by the target language. L2 checks the corresponding results on the model and artifacts. For adaptations that generate the target language directly, a domain parser can load the model. Under either path, output completion, model construction, and domain validation remain explicit steps. Incomplete output cannot pass acceptance checks.

Hosted structured-output interfaces enter this process through schema submission and validation of the returned values. A local backend under direct control additionally allows observation of constraint interventions at individual decoding steps. Both use the same relationship between generation constraints and result checks. The corresponding experiments describe how the decoding process is observed.

3.6 Post-generation validation and task acceptance

L2 checks whether the candidate model and serialized artifacts satisfy the metamodel, applicable domain rules, and the current task. It divides responsibility with L1 according to expressive capabilities and contextual requirements. Requirements that can be expressed in advance as structural restrictions or finite value choices are enforced during generation. Requirements that depend on complete objects, relationships across objects, or target artifacts are evaluated after construction. The same rule can participate in both layers. For example, L1 filters reference candidates by target type, while L2 checks whether the constructed references resolve correctly. Recent work also explores semantic checks during model construction [21]. The

two layers therefore do not represent a fixed division between syntax and semantics.

Validator construction and integration. During domain preparation, the integrator uses ICM constraints and their sources to determine the input objects, applicability conditions, decision logic, and diagnostic locations for each check. Properties supported by existing tools are checked through their interfaces, as in target format validation. Properties expressible through existing check operations are implemented using rule templates and parameter configurations. Dedicated checkers handle consistency requirements across objects. In this work, custom checkers are developed by humans with LLM assistance. The LLM helps organize rules and draft code and tests. Humans consult the specifications to confirm rule interpretations, select implementations, and review the decisions produced by the checks.

Checkers are tested using examples that satisfy or violate the rules, together with relevant boundary cases. Their rule identifiers, scopes, inputs and parameters, results, and diagnostic information are then registered. Task execution selects the required checks from these registered capabilities and binds them to the current objects and context. Temporary constraints derived from natural-language tasks also enter the current check configuration through existing check operations and parameters. When new decision logic is needed, the domain integrator adds the corresponding checking capability.

Validation targets and acceptance criteria. Validation is organized around four aspects of the objects being checked. These are the format validity of the generated representation and target artifacts, the **meta-model conformance** of the candidate model, domain **well-formedness constraints** and other rules, and **requirements satisfaction** for the current task. Meta-model conformance covers types, attributes, relationships, and multiplicities. Well-formedness constraints can address consistency across elements, while some format tools also check content structure [3]. The check configuration specifies the tool or checker responsible for each requirement.

Task acceptance is based on the original requirements or a structured task specification derived from them. Merely checking whether an artifact conforms to the LLM-proposed design cannot reveal requirements omitted from that design. For example, if the design incorrectly specifies 20 ms instead of 10 ms, the generated result may conform fully to the design. Acceptance must still identify the discrepancy against

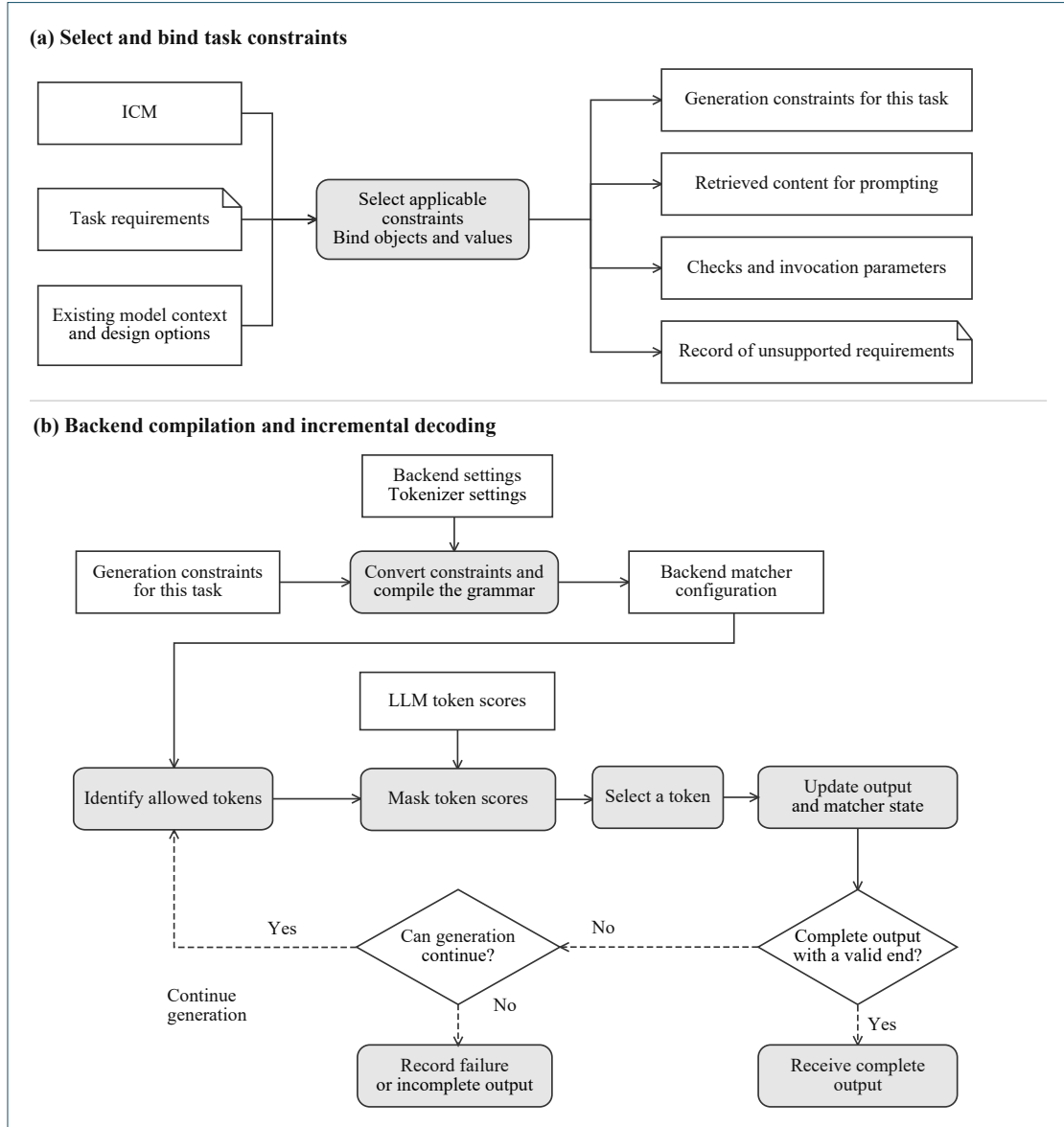


Fig. 4 Task binding and L1 enforcement. (a) Task requirements, the ICM, and existing model context are used to assemble generation constraints, retrieved content, and check configurations. (b) Grammar compilation, allowed-token filtering, and state updates. Complete output enters L2 after structural validation and model assembly

the original period requirement. Similarly, deleting a required timing event may remove a reference error without providing the periodic triggering required by the task.

Required checks and their applicability conditions are determined before generation. The task is accepted within the corresponding scope when all applicable checks have completed and passed. Rule violations enter failure diagnosis, while required judgments that cannot be completed retain an incomplete status. The

validation report records rules, object locations, and results so that repair can address specific problems and the final conclusion corresponds to the checks actually performed.

3.7 Model repair with validation feedback

Model repair relies on locating inconsistencies, defining edit operations, and identifying repair candidates [23]. The method uses rule identifiers, model elements, and artifact locations from L2 to identify

objects that may be modified. It supplies the LLM with failure diagnostics, relevant rules, the original task requirements, and permitted operations. The domain integrator defines these operations, such as changing an attribute value, replacing a reference, adding a required object, or removing an element whose deletion is permitted.

After the LLM proposes structured changes, a deterministic procedure checks whether the operations and their targets are permitted, applies the changes to the model or task execution state, reconstructs the artifacts, and runs the relevant validation. Revalidation checks domain rules, the original task, and protected content together. Only changes that meet the editing restrictions and acceptance conditions are carried forward into the subsequent state. Repair succeeds when the task passes acceptance checks again.

The repair loop has an attempt limit and retains the changes and revalidation results. It terminates when acceptance checks pass. If the limit is reached without acceptance or the problem exceeds the available editing capabilities, the current model and diagnostics are retained for human handling. If a person changes the original requirements or domain rules, task bindings, generation constraints, and check configurations must be updated before the corresponding construction and validation steps are repeated. Figure 5 shows the relationship between validation, repair, and revalidation.

3.8 Domain adaptation and traceability

The interfaces in Table 3 connect the general process to domain implementations. The ICM provides a common way to organize constraints, while adaptation components provide metamodel queries, generation mappings, model construction, validation, and editing capabilities. Adapters call existing domain tool interfaces for supported operations. Other operations are implemented according to the target metamodel and artifact representation.

Traceability records connect task requirements, ICM rules and sources, the generation constraints actually used, models and artifacts, validation diagnostics, and modification results. Domain rules are associated with task objects through metamodel elements, while binding records connect task requirements to concrete values and relationships. These records allow a restriction to be traced to its source and show the stage in which it was enforced, the object to which it applied, and whether it was satisfied after repair. Temporary

task constraints are retained with the run, and their scope remains limited to that task.

This adaptation structure combines metamodel-guided constraint organization, L1 enforcement, and L2 validation into a reusable method. Model representations, rule content, and concrete execution capabilities vary across domains. The shared layered principle is to restrict permitted content during generation, check complete results afterward, and accept repairs against the original task requirements.

4 Empirical evaluation

This section evaluates the role of the layered constraint method in model generation and repair. The AUTOSAR experiment uses the prepared ICM to examine model instance generation, ARXML artifact acceptance, and repair of injected faults. The AUTOSAR-vLLM experiment directly observes how generation constraints intervene in token selection. The railway experiment examines whether validation feedback can support the repair of models with relationships across objects. The private international law experiment examines layered checks in structured legal decision-making. Appendices A through D follow the same order and provide additional settings, detailed results, examples, and the statistical methods used for the main comparisons. Complete paired data, sensitivity analyses, and calculation scripts are included in the replication package.

4.1 Research questions and evaluation setup

The evaluation addresses three research questions.

- **RQ1. How can constraints from specifications and metamodels be applied to an individual model generation task?** The AUTOSAR case study demonstrates source records, associations with metamodel elements, task binding, and constraint execution through the ICM, and checks the generated artifacts.
- **RQ2. Do generation-time constraints actually change the decoding process?** AUTOSAR-vLLM compares constraints provided only in prompts with constrained decoding under the same prompts. It records where constraints exclude candidate tokens and examines the resulting structure.

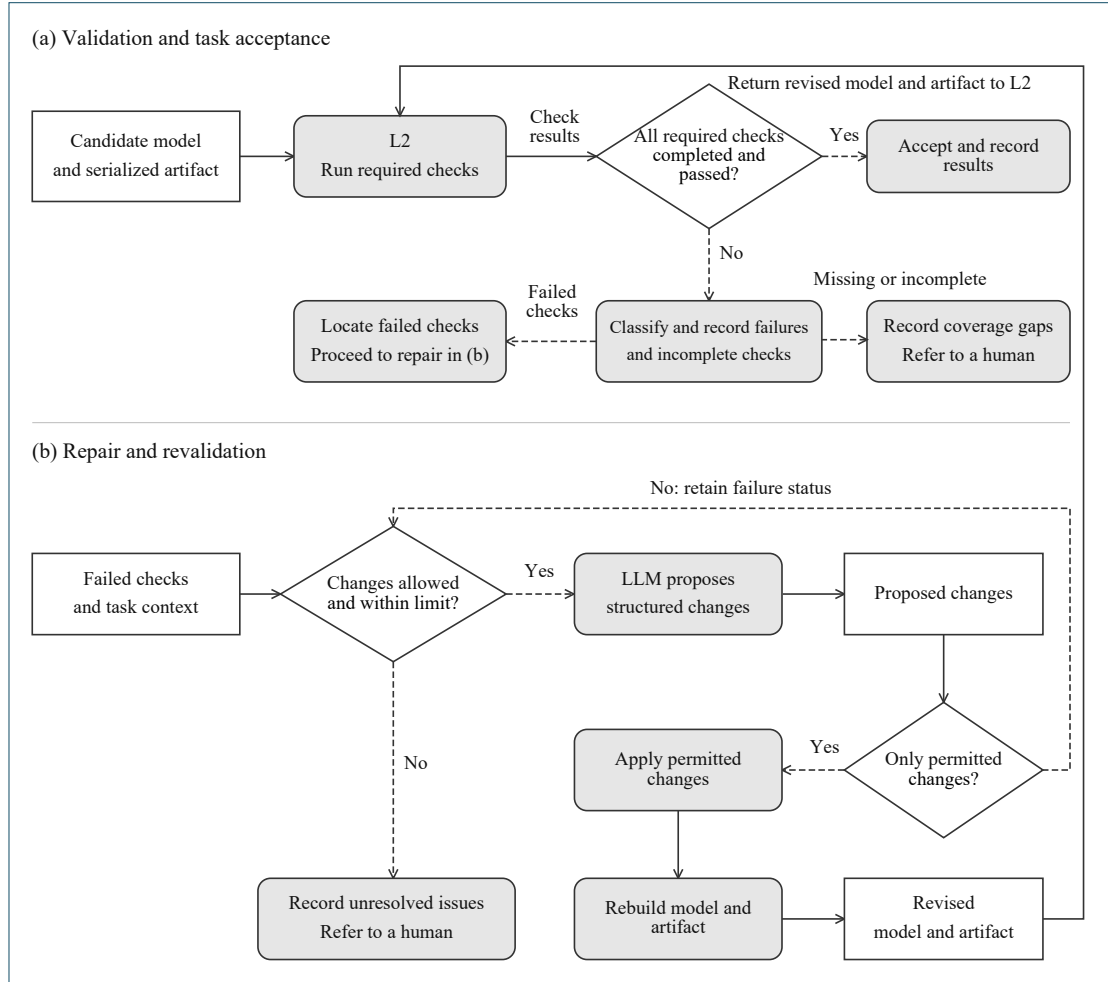


Fig. 5 L2-guided repair and revalidation. Failure diagnostics locate the objects to modify, and permitted operations limit the repair scope. Changes are followed by reconstruction and revalidation, with the original task requirements retained throughout acceptance. Problems unresolved after automated processing are referred for human handling

Table 3 Domain adaptation interfaces

Interface	Inputs	Outputs and responsibilities
Metamodel transformation and querying	Existing metamodel and language mappings	Queryable representation, source element correspondences, and identifiers
Constraint selection and assembly	ICM, task requirements, model context, and designs from preceding stages	Generation constraints, retrieved content, checks, and parameters
Model construction and serialization	Generated content and assembly information	Model objects, containment and references, and target artifacts
Validation invocation	Models and artifacts, check configurations, and context	Results and diagnostics linked to rules and object locations
Issue location and modification	Failures and permitted operations	Checked model changes and inputs for revalidation

- **RQ3. Can post-generation validation and repair correct errors in models or decisions while preserving the original task requirements?** The AUTOSAR experiment examines the detection and repair of engineering faults. The railway experiment compares repair strategies starting from the same initial model and includes a second LLM. The private international law experiment examines the effects of auditing, revision, and release control on decision outputs.

The primary outcome in the AUTOSAR and railway experiments is whether a run meets the **strict success criterion**. The final artifact must pass the applicable structural checks, domain checks, and acceptance checks against the original task. For example, three runs of a railway task yield three initial generations. For each initial generation, self-repair and repair with validation feedback start from the same initial model. Success rates use these 72 initial generations as the denominator. An attempt that produces no model or still fails after repair is counted as a failure. The number of base tasks remains 24, representing the diversity of requirements and layouts. The conditions for each experiment are defined when first introduced in the corresponding subsection.

The main experiments with hosted models used gpt-5.6-luna with low reasoning effort. The second LLM in the railway experiment was gpt-5.6-terra, also with low reasoning effort. AUTOSAR-vLLM used a local Qwen3.5-9B model to obtain decoding traces. Custom validators were written by humans with LLM assistance. The authors interpreted the rules, determined the inputs and applicable objects, selected the check implementations, and tested them. Existing XSD, EMF, and VIATRA validation capabilities were reused directly. The construction and configuration of checks for each domain are described in the corresponding subsections. The natural-language inputs for AUTOSAR and PIL generation tasks were in English. The railway experiment used structured task specifications written by the authors, including English task descriptions.

4.2 Constraint organization and engineering model generation in AUTOSAR

4.2.1 Requirement sources and generation workflow

AUTOSAR describes automotive software components, interfaces, and their interactions. ARXML is the corresponding XML exchange representation. The authors designed 20 requirements concerning component structure, ports, and references based on the Software Component Template of AUTOSAR Classic 4.2.2 [28]. The meaning of periodic triggering and explicit data access was checked against the Specification of RTE from the same release [29]. The case combinations also drew on the model structure of an RH850 project in an ETAS toolchain. The authors removed the original project context from the observed interfaces, ports, runnable entities, timing events, and access relationships, then decomposed and recombined these elements and varied their parameters. This process produced 6 minimal, 7 standard, and 7 full cases. The specification sources and the content designed by the authors are documented in Appendix A.1 and Table A1.

Minimal cases focus on a single port and its interface. Standard cases combine multiple ports with periodic execution behavior. Full cases further combine multiple runnable entities, periods, and access relationships. In addition to the requirement description, the inputs include a structured case specification and a catalog of existing interfaces. Task analysis binds requirements to specific objects. These bindings are combined with the metamodel information, structured constraints, and model context in Neo4j to assemble the JSON Schema for the current task. The LLM generates structured content under these constraints. The adapter assembles the model and serializes it as ARXML in the element order specified by XSD.

Constructing these models involves containment hierarchies, collection cardinalities, and references across files. The XML elements in the evaluated ARXML batch have a maximum nesting depth of 14, counting the root element as level 1. Structured output and the serializer therefore jointly construct the model representation. XSD, reference, and task checks are then applied to the actual ARXML artifacts.

Table 4 Experimental design and scale

Experiment	Base tasks and runs	Main evaluation targets
AUTOSAR engineering model generation	20 requirements with 3 runs each; an additional 85 core and 15 substitute fault-injection units	ICM use, ARXML structure and task satisfaction, controlled repair
AUTOSAR-vLLM decoding comparison	The same 20 cases with 3 runs each under three conditions, totaling 180 generations	Token-level interventions, structural acceptance, auditing overhead
Railway model generation and repair	24 base tasks with 3 runs each; 1 run per task with the second LLM	XMI models, domain relationships, and original task requirements
Private international law decisions	60 scenarios with 3 runs each under four conditions, totaling 720 outputs	Checks and revisions of structured decisions, and compatibility with reference judgments

4.2.2 From the ICM to an individual check

The AUTOSAR ICM contains 1,085 structured constraints and 6,533 constraint associations. The validation plan contains 554 checking rules. Checks are executed at runtime according to the generated model and the applicability conditions of the rules. Constraint extraction from specifications, metamodel linking, and validator construction are described in Appendix A.2, with the extraction setup summarized in Table A2. Figure 6 illustrates the use of a constraint on periodic runnable execution. Specification rule TPS_SWCT_01519 is associated with runnable entities, internal behavior, and TimingEvent [28]. Task binding selects the specific runnable entity. At runtime, an existing timing-event check determines whether the event exists, whether it targets that runnable entity, and whether its period value can be parsed. The single applicable object in this check passed validation.

The existence of a period does not by itself establish that the task is complete. For example, the task specifies a period of 10 ms. The acceptance checker reads this expected value from the case requirements and compares it with 0.01 s in the ARXML. Even if the generation plan contains an incorrect period, agreement between the artifact and that plan alone cannot satisfy task acceptance. The ICM preserves the specification basis and applicable metamodel elements. Task binding determines the object to be checked, and task acceptance checks the specific value required for the current task.

4.2.3 Generation results and execution cost

Each of the 20 requirements was used in three runs. All 60 workflow outputs passed acceptance checks for the specified component tasks, producing 255 ARXML

files in total. Table 5 summarizes the artifacts, checks, and execution cost. These results cover the complete workflow, including task analysis, constrained generation, deterministic assembly, serialization, and validation.

Table 5 Results of the complete AUTOSAR generation workflow

Metric	Result
Generation runs passing task acceptance	60/60
ARXML files passing XSD validation	255/255
Case structural obligations checked	2,418, all passed
Local references checked	675, all passed
Recorded workflow token usage	11,967,476
Mean time per workflow run	64.712 s

The controlled repair experiment separately injected five types of faults into deterministic reference models that had passed acceptance checks. The faults were empty initial values, incorrect initial values, missing event or internal-behavior paths, missing receiver communication properties, and incorrect XSD element order. Faults were detected in all 85 applicable units, and the affected files were restored to the reference content within one actual repair round, using 661,631 tokens in total. Some minimal cases lack receiver ports or internal behavior, so the corresponding faults could not be injected. Before execution, reference, port-name, or component-name faults were designated for these positions, forming 15 substitute units. All of these were also restored. Results for the core and substitute faults are reported in Appendix A.3 and Tables A3 through A4. Appendix A.4 uses actual

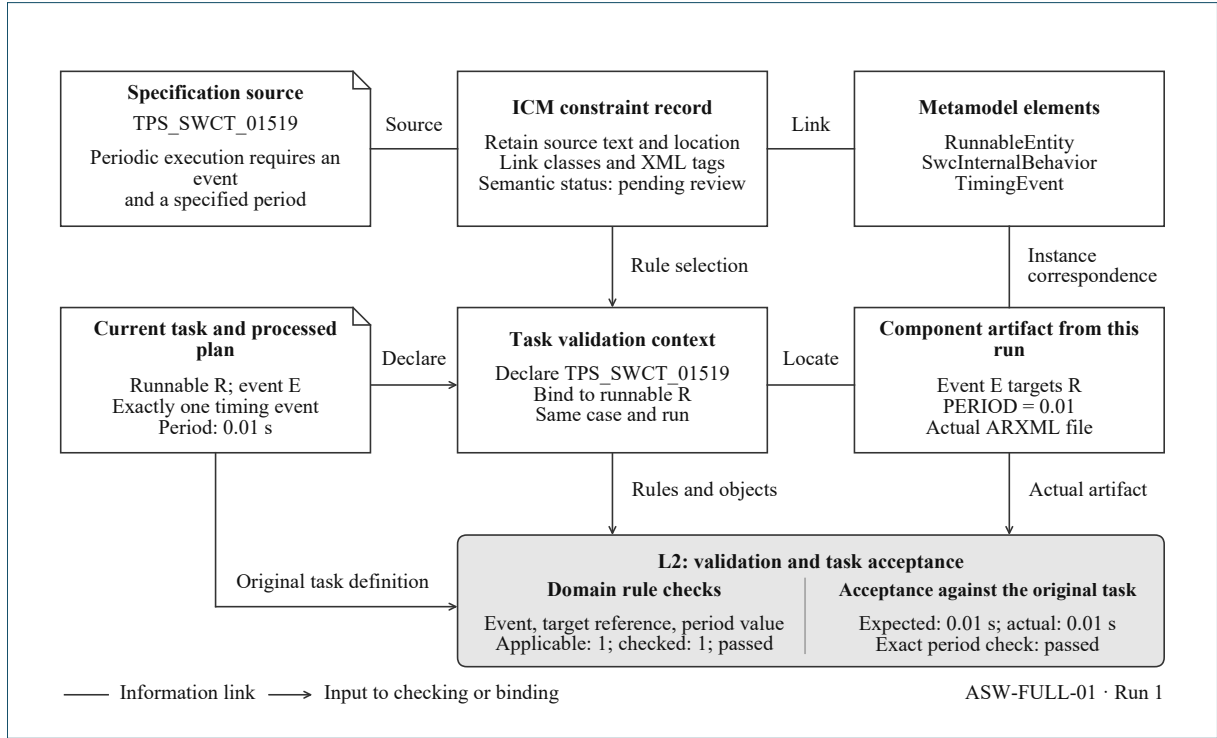


Fig. 6 Links from the specification source of a periodic runnable constraint to model objects and check results. R denotes RE_Com_Full_Baseline, and E denotes TE_Com_Full_Baseline. The required period of 10 ms is represented as 0.01 s in the artifact

ARXML fragments to illustrate the generation, acceptance, and independent controlled repair of a periodic component.

The AUTOSAR generation results address RQ1. Constraint sources, metamodel associations, and task bindings can be connected to actual generation and checking, producing component models with multiple containment levels and interrelated references. Controlled repair provides further engineering evidence for RQ3. Validation localizes faults in structure, values, and references. After repair, the artifacts pass revalidation and recover the task properties satisfied by the reference artifacts.

4.3 Decoding interventions by generation constraints in AUTOSAR-vLLM

This experiment fixes the AUTOSAR cases and their generation constraints to observe the role of L1 directly during generation. It compares three conditions. Under **prompt-only constraints (U)**, the requirements and complete JSON Schema are provided only in the prompt. **Constrained decoding (G)** additionally enables decoding constraints with the same prompt. **Constrained decoding with auditing**

(A) records stepwise decoding information on top of G. All three conditions use the same model and sampling settings, with execution order balanced across conditions. The experiment uses vLLM [30] and XGrammar [19].

Constraints were observed to intervene in token selection in all 60 audited generations. In each generation, grammar constraints excluded the highest-scoring unconstrained token at 7 to 10 steps. Across runs, this occurred at 516/54,942 evaluable steps, or 0.9392%. Figure 7(a) shows the intervention positions for each case. The traces of the three runs for the same case coincide exactly and are combined into one row. This shows that generation constraints narrow the set of available tokens during decoding, rather than merely checking format correctness after output completion. The proportion measures steps at which the local token choice was directly affected. It does not measure the proportion of the final output content influenced by the constraints.

All three conditions produced parseable JSON. Structural acceptance was achieved by 45/60 outputs under prompt-only constraints and by 60/60 outputs under both constrained decoding and auditing. Failures under prompt-only constraints were concentrated in

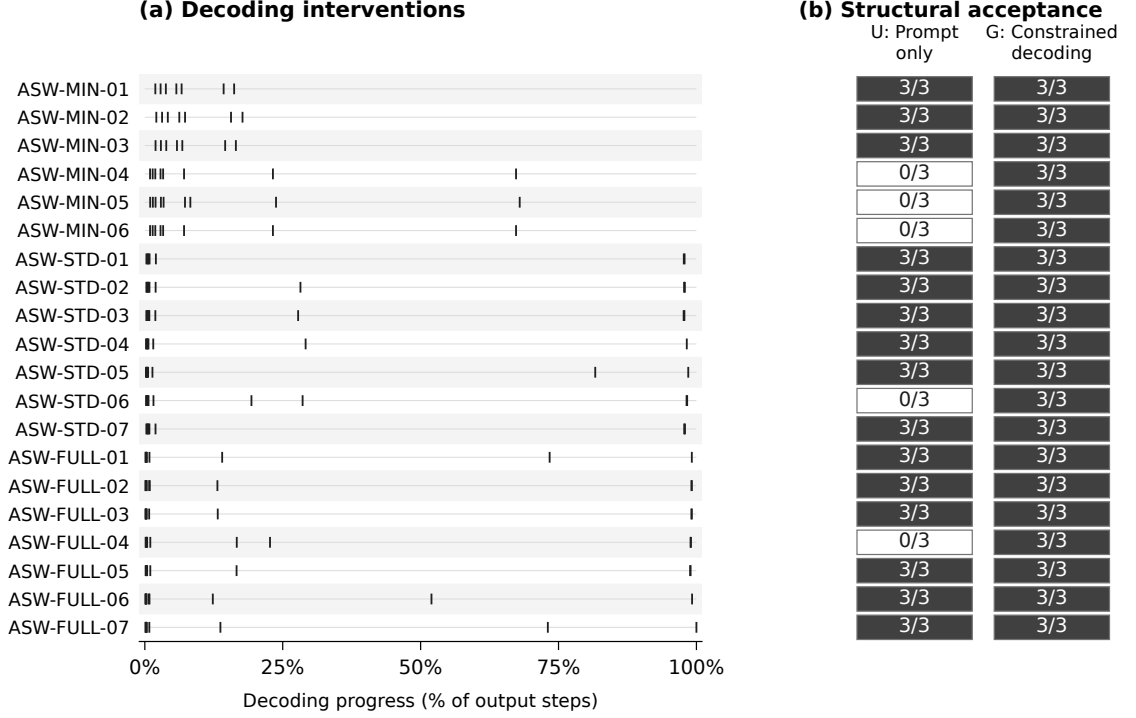


Fig. 7 AUTOSAR-vLLM decoding and output results. (a) The intervention traces of the three audited generations for each case coincide exactly and are combined into one row per case. Each tick marks a step at which grammar constraints excluded the highest-scoring unconstrained token. (b) The number of runs passing structural acceptance under prompt-only constraints and constrained decoding, with three runs per case

five cases and involved arrays being written as objects and incorrect nesting of access references. These errors show a substantive difference between parseable JSON and a model representation that conforms to the task structure. With generation constraints enforced, all three runs of each of these five cases passed the corresponding structural checks and subsequent artifact checks.

The 60 paired outputs from the auditing and constrained decoding conditions were identical byte for byte. The mean request time was 10.365 s with constrained decoding and 13.252 s with auditing added. Total request time increased by 27.85%. Observing the generation process therefore incurs an additional execution cost. These results address RQ2. L1 intervened in decoding and eliminated the structural violations observed under prompt-only constraints in this case set. Execution settings are given in Appendix B.1. Results at the different acceptance stages and request costs are reported in Appendix B.2 and Tables B1 through B2.

4.4 Post-generation validation and bounded repair of railway models

4.4.1 Tasks, validators, and comparison conditions

The railway experiment uses the existing metamodel and six model queries from the Train Benchmark [31]. The authors designed three types of configuration tasks concerning service connections, maintenance preparation, and redundant monitoring. Each task type was applied to eight railway layouts, producing 24 base tasks. Structured task specifications declare fixed objects, physical connections, assignable fields, and task obligations. This information is compiled into structural plans and generation constraints. The LLM selects field values, and an adapter assembles the candidate model and outputs XMI.

Post-generation checks load the XMI directly. EMF model diagnostics check structure and references, six VIATRA queries check domain relationships, and further checks assess task obligations and fields that must remain unchanged. For example, disabling a route that must remain active may reduce violations of a domain

rule, but task acceptance still rejects the change. The six native queries and their check implementations are described in Appendix C.1 and Table C1. Repair conditions and the permitted scope of changes are given in Appendix C.2.

Each task was run three times, yielding 72 initial generations. The comparison covers **initial generation (G0)**, **self-repair (GS)**, and **repair with validation feedback (GF)**. Self-repair asks the LLM to check and revise its own output against the rules, following an approach related to the self-revision idea in Self-Refine [24]. Repair with validation feedback additionally provides actual diagnostics, violation locations, and local candidates, while acceptance checks preserve requirements already satisfied. Each repair branch permits at most two rounds, with changes to at most six explicit fields per round. Organizing repair around the causes of inconsistencies and restricting the scope of changes also relates to existing model repair research [23].

4.4.2 Main results and failure analysis

As shown in Figure 8(a), the strict success rate increased from 23/72 after initial generation to 29/72 after self-repair and 51/72 after repair with validation feedback. The latter represents an increase of 38.89 percentage points over initial generation. Paired success-rate differences on the same tasks and their intervals are reported in Appendix C.2 and Table C3. Among the 48 failed initial generations that produced models, self-repair recovered 6 and repair with validation feedback recovered 28. One other initial generation produced no model because its initial assignments did not meet the acceptance conditions.

The 21 failures after repair with validation feedback have identifiable causes in the process. One initial generation produced no model. The remaining 20 models still violated SemaphoreNeighbor, the rule governing relationships between boundary signals. Of these, 16 proposals were rejected because they caused previously passing checks to fail, the LLM abstained in 3 cases, and 1 proposal was rejected because it removed no violations. Validation can therefore identify problems and prevent unacceptable changes, but the current candidate selection and repair strategies may not find changes that satisfy interdependent requirements simultaneously. The result of 51/72 reflects the observed improvement achieved through accepted repairs. Repair success remains affected by the candidate space and the LLM’s ability to propose changes.

Appendix C.4 uses actual XMI fragments to illustrate diagnosis of relationships across objects, repair of a single field, and revalidation.

A supplementary experiment compared repair configurations that progressively add validation feedback and information about violation locations. On the same inputs with injected faults, it compared self-repair, the addition of validation feedback, and the further addition of local candidates and dependency information. The corresponding numbers of successful runs were 129/144, 140/144, and 141/144. Adding local information produced one net additional success relative to validation feedback alone. Four units succeeded only with the additional information, while three succeeded only without it, indicating that the benefit varied with the fault conditions. The main text uses repair following initial generation as the principal evidence. Results for controlled faults, faults in task obligations, and preservation of correct models are given in Appendix C.2 and Table C2.

4.4.3 Evaluation with a second LLM

On the same 24 tasks with the fixed seed 104729, Luna achieved 6, 8, and 13 successful runs for initial generation, self-repair, and repair with validation feedback, respectively. Terra achieved 18, 22, and 22, as shown in Figure 8(c). Terra had more successful initial generations, and each of its two repair branches recovered four additional tasks. However, the final sets of successful tasks were not identical. The two tested model configurations exhibited different initial errors and opportunities for repair. A higher initial success rate does not necessarily lead to a greater benefit from validation feedback over self-repair. Settings for the second LLM, results by condition, and token usage are given in Appendix C.3 and Table C4.

4.5 Layered checks for structured legal decisions in private international law

Private international law (PIL) addresses applicable law, jurisdiction, and related issues involving cross-border elements. This experiment focuses on jurisdiction decisions to examine the use of layered constraints in a decision domain requiring high reliability. Output structure constraints ensure that fields such as conclusions, jurisdiction types, and legal grounds can be processed. Post-generation auditing then checks cross-field relationships and citations and triggers revision.

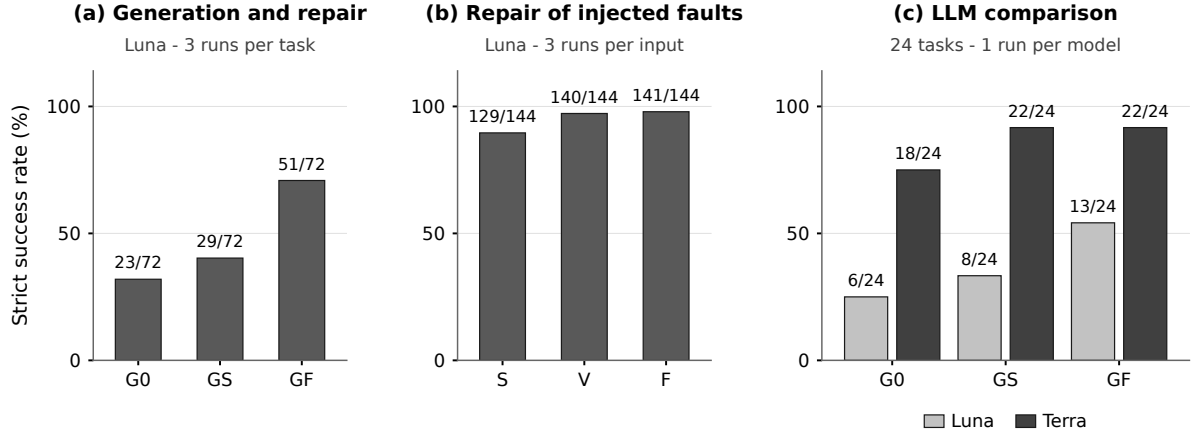


Fig. 8 Railway model results. (a) Two repair strategies applied to 72 initial generations. (b) Self-repair, repair with validation feedback, and repair with additional information about violation locations on 144 inputs with injected faults. (c) Comparison of Luna and Terra on the same 24 tasks with a fixed seed

The outputs are structured legal decision records. Serialization and native model checks used for engineering models are replaced by checks of the decision structure and its encoded relationships.

Two graduate students in private international law constructed the case set based on Brussels I bis [32] and the EU Trade Mark Regulation [33]. The set contains 60 synthetic scenarios covering general and special jurisdiction, jurisdiction agreements, procedural relationships, and referral to national law. The students independently reviewed the reference labels, without seeing each other’s judgments in the initial round, and resolved disagreements through discussion. The reference judgments were established before the experimental generation runs.

The experiment compares four progressively augmented conditions. **Basic prompting (P0)** provides only the common output requirements. **Retrieval augmentation (P1)** adds legal grounds. **Structural constraints (P2)** further enforce a strict output structure. **Full checks (P3)** add post-generation auditing, one repair attempt, and release control. Retrieval augmentation follows the idea of retrieval-supported generation discussed in the related work [17]. Each condition includes three runs for each of the 60 cases, producing 180 outputs. Case preparation and the inputs, generation, and processing conditions are described in Appendix D.1 and Table D1.

The primary endpoint requires a valid decision structure, satisfaction of the common delivery rules, compatibility with four requirements derived from the

reference judgments, and actual release of the output. These reference requirements concern the conclusion, primary jurisdiction type, required provisions, and conditional-answer flag. Table 6 presents the main results. Appendix D.2 and Table D2 further distinguish valid delivery, actual release, and reference compatibility.

Full checks improved the primary endpoint by 23.3 percentage points over structural constraints. The 95% resampling interval based on paired cases was [13.3, 33.9] percentage points. All four comparisons are reported in Appendix D.2 and Table D3. Repair was triggered for 62 outputs. Of these, 40 changed from incompatible to compatible with the reference judgments and were released, while 3 changed from compatible to incompatible. These results show that, even after structural correctness is achieved, relationships between fields may still require post-generation checks. They also show the need to repeat acceptance checks after repair.

PIL provides application evidence for RQ3 in a decision domain. The same division between generation-time restrictions and post-generation checks can be used for structured decisions. Free-text legal reasoning still requires separate evaluation. Targeted expert review found that an answer passing the automated metrics could nevertheless confuse the jurisdiction of courts in a Member State with that of courts in a specific city. Actual JSON field revisions and revalidation are illustrated in Appendix D.3. The counterexample and expert judgments are given in Appendix D.4.

Table 6 Main PIL results

Condition	Valid structure	Primary composite endpoint
Basic prompting (P0)	180/180	73/180 (40.6%)
Retrieval augmentation (P1)	178/180	96/180 (53.3%)
Structural constraints (P2)	180/180	101/180 (56.1%)
Full checks (P3)	180/180	143/180 (79.4%)

4.6 Threats to validity

The conclusions are limited by the scope of the tasks and checks. AUTOSAR evaluates component modeling, the railway experiment evaluates configuration models and six existing queries, and PIL evaluates structured decisions against defined reference requirements. Free-text explanations and system behavior after deployment are outside the scope of these checks. Although the custom validators were tested, rule interpretations and task specifications may still reflect bias in human judgments. Some specifications are used in both generation and acceptance, which may share the same bias. We did not separately measure person-hours for domain preparation, checker development, or human intervention after failure. Model response times and token usage reflect only automated execution costs.

The diversity of the cases and the comparison conditions affect generalizability. AUTOSAR requirements draw on specifications and one engineering example. Railway tasks combine three task types with eight layouts, share templates, and were available during development. The PIL support library and retriever were developed on the case set used in the experiment. Three repetitions allow observation of variation across runs of the same task. Some repair conditions change both the feedback and the policy for accepting modifications, so their effects cannot be attributed entirely to feedback content. The second LLM was evaluated with only one seed. Further evaluation should cover more independent tasks, rules, and LLMs, and separately examine constraint extraction quality, validation coverage, and repair strategies.

5 Conclusion

This paper presented a metamodel-guided method for model generation with layered constraints, combining natural-language interaction with explicit model representations and validation capabilities in MDE. Starting from an existing domain metamodel, the method extracts and transforms metamodel information, uses

metamodel terminology to guide structured constraint extraction, and automatically links constraints to metamodel elements. The resulting ICM retains rules, element associations, and specification sources. For each task, constraints are bound to specific objects, values, and references, providing the basis for execution in the generation-time constraint layer (L1) and the post-generation validation layer (L2). L1 controls the structures available during generation, L2 checks models and their artifacts, and the original task requirements govern candidate acceptance and subsequent repair throughout the process.

This division of responsibilities combines model construction under structural constraints with logical validation. Requirements on mandatory features, multiplicities, and available references can restrict generation when suitable representations and execution capabilities are available. Checks that depend on complete objects, relationships across elements, or target artifacts are performed after model construction. The ICM connects these operations to their domain sources, allowing the same rule to support generation constraints, retrieval for prompts, and validator development. It thereby supports constraint reuse from domain preparation through task execution.

Experiments examined the method in three domains. All 60 end-to-end AUTOSAR runs passed acceptance checks within the specified task scope, and all 255 generated ARXML files passed XSD validation. All 85 core units and 15 substitute units in the controlled fault-injection experiment were restored to the reference artifacts. A separate local AUTOSAR experiment recorded interventions by constraints during decoding in all 60 audited generations. In the main railway experiment, 23 of 72 runs met the strict success criterion after initial generation, compared with 51 after repair with validation feedback. This indicates that validation feedback helped repair a substantial proportion of failed candidates, while some tasks remained incomplete under the current repair configuration. The PIL experiment applied structural

restrictions, rule checks, and evaluation of task conclusions to decisions in private international law, providing evidence for the layered principle beyond engineering models. Together, the experiments show the roles and practical limits of constraint enforcement, acceptance checks, and repair.

The current method depends on metamodels, generation mappings, and validation capabilities supplied during domain adaptation. The mapping from models to artifacts is also an essential part of the process. For example, when model features are represented as JSON object properties, an XSD-compliant serializer must enforce the element order required by ARXML. Validation of the intermediate representation alone cannot establish the validity of the target artifact. Future work should systematically evaluate the completeness of mappings between intermediate representations and target formats and move more incrementally checkable requirements into L1. Repair also requires further study of error localization, search over effective modifications, and the treatment of candidates that cannot be completed under the current configuration, while preserving the original task requirements.

Recent work on LLMs and MDE has addressed model and metamodel construction [6], semantically guided generation [21], and formal specification generation [25]. As natural language is increasingly used to construct models, specifications, and code, explicit modeling languages, model transformations, and validation tools remain important foundations for automation. The layered principle in this paper provides a methodological basis for coordinating these capabilities. Restrictions that can be decided early are enforced during generation, properties requiring complete context are checked afterward, and the final artifact is assessed against the original requirements. This basis can be extended to the preparation of domain resources.

The first direction is **generating domain metamodels from natural language**. Research could identify concepts, attributes, relationships, multiplicities, and well-formedness requirements from domain specifications and representative requirements. It should distinguish domain concepts shared across tasks from the objects and values of an individual task, producing candidate metamodels with links to their sources. Evaluation should examine the appropriateness of conceptual abstractions, consistency of terminology interpretation, and ability of the modeling language to express the intended tasks. Requirements not used

during construction, independently developed model instances, and review by domain experts could be used to assess coverage and omissions. Once validated, the metamodel could serve as input to ICM construction.

The second direction is **generating formal verification artifacts from natural language**. Research could translate specification requirements into formal specifications with explicit types, applicability conditions, and contextual assumptions. Together with structural or behavioral representations of the system under verification, these specifications could support the construction of verification models, proof obligations, and corresponding checking implementations. Evaluation should separately examine whether the requirements have been faithfully formalized, whether the checking implementations conform to the defined properties, and what conclusions the tools can establish under the given models and assumptions. Validated properties and implementations could be integrated into L2, while those that can be decided during generation could also be mapped to L1.

Both directions must address the risk that shared interpretation errors reinforce one another during automation. A metamodel, candidate models, and their checking criteria may be mutually consistent while omitting the same original requirement. Independent semantic references, counterexamples, and expert review can help assess the quality of domain resource preparation and model instance generation separately. Source links in the ICM and the explicit division of responsibilities between L1 and L2 provide a traceable basis for extending these capabilities and identifying further research problems in model generation and validation.

Data and code availability

The research data and code are available in the replication package. They include domain constraints and their sources, tasks and prompts, generated artifacts, validators, repair records, and statistical analysis scripts. The repository README provides navigation in the order of the experiments in Appendices A through D, together with a complete offline package and file checksums.

Appendices

The appendices follow the order of the experiments in the main text. Appendix A provides further details

on AUTOSAR requirement sources, the ICM and validators, controlled repair, and an engineering example. Appendix B reports the settings and costs of the AUTOSAR-vLLM decoding comparison. Appendix C supplements the railway rules, comparisons, and repair example. Appendix D describes PIL scoring, expert assessments, and the revision of an actual decision. Appendices A.4, C.4, and D.3 each use actual ARXML, XMI, or JSON fragments to illustrate the task, generation, diagnosis, repair, and revalidation. The units of analysis, aggregation of repeated runs, and resampling settings for the main comparisons are described in C.2 and D.2. Complete paired data, statistical scripts, and sensitivity analyses are available in the replication package.

A AUTOSAR model generation and repair

A.1 Requirement design and generation settings

The 20 requirements were designed using the Software Component Template and the Specification of RTE for AUTOSAR Classic 4.2.2 [28] [29]. The former defines components, ports, communication specifications, internal behaviors, and reference structures. The latter informs the interpretation of periodic triggering and explicit data access. Combinations of interfaces and behaviors observed in an RH850 project using the ETAS toolchain provided engineering context. The authors removed project-specific context, normalized names, decomposed and recombined elements, and varied parameters to construct the three tiers in Table A1.

The normative basis for the initial value is `constr_1201` in the Software Component Template, with its usage explained by `TPS_SWCT_01220` [28]. The value 0 is a specific task-design choice. Together, the 20 base cases require 33 provided ports, 32 required ports, 18 runnable entities, 18 timing events, and 55 variable accesses. Detailed case provenance and design descriptions are included in the replication package.

Generation used `gpt-5.6-luna` with low reasoning effort, a temperature of 1.0, strict structured output, and a maximum output budget of 128,000 tokens. The three fixed seeds were 104729, 130363, and 155921. Task inputs comprised an English requirement description, a structured specification derived

from that requirement, and an existing interface catalog. Task analysis selected the required structure. Generation constraints were then assembled using these selections, the ICM, and metamodel information in the graph. The adapter bound fixed objects and values, assembled the structured output, and serialized it as ARXML. Generation evaluation and controlled repair were conducted separately.

The 60 generation runs produced 60 component files and 195 interface files. Individual component documents contain 11–115 XML elements, and each interface document contains 10 elements. The maximum nesting depth is 14, counting the AUTOSAR root as level 1. One output for the largest case, ASW-FULL-06, contains seven documents with 175 elements in total. These counts refer to XML element nodes. Attributes, text, and repeated copies are not counted separately.

A.2 Constraint extraction, ICM, and validators

Domain preparation for AUTOSAR used the text of Chapters 2–13 of the Software Component Template and the associated metamodel information [28]. The authors annotated chapters, relevant classes, and enumerations. The system added the corresponding attributes and enumeration literals from the transformed metamodel representation and organized contextual input fragments by chapter and paragraph. The LLM extracted structured constraints and their target elements from these inputs. The linker automatically checked target classes, attributes, and enumeration literals and established references. Records with unresolved links, member mismatches, or similar issues were corrected by humans and checked again. Table A2 summarizes this preparation process.

The model generation experiment used fixed ICM resources containing 1,085 structured constraints, 6,533 constraint associations, 11,177 structural nodes, and 14,106 structural relationships. The ICM retains specification provenance and links to metamodel elements. Task execution uses this information to select relevant constraints and bind specific objects.

The validation plan contains 554 checking rules, comprising 400 Python backend rules and 154 declarative rules. Checks are executed at runtime according to the generated model and each rule’s applicability conditions.

Table A1 Design and sources of the AUTOSAR requirements

Content	Source	Design procedure
Interfaces and ports	Three provided interfaces, three required interfaces, and their data elements in the engineering project	Recombine interfaces and normalize names
Period and reception timeout	Engineering baseline values of 0.01 s and 0.3 s, respectively	Vary period and timeout parameters
Initial value	constr_1201 requires an initial value for the selected nonqueued receiver communication specification	Use the author-selected value 0 throughout
Minimal tier, 6 cases	A single port and its interface	Construct either a provided port or a required port
Standard tier, 7 cases	Ports, periodic execution, and access relationships	Combine 2–3 ports, one runnable entity, and a timing event
Full tier, 7 cases	Multiple interfaces and internal behaviors	Combine six ports, 1–3 runnable entities, and period variations

Table A2 Setup for AUTOSAR constraint extraction and linking

Item	Description
Source material	Identified provisions, normative descriptions, and relevant examples in the selected chapters, with provision identifiers and chapter provenance retained
Terminology context	The fragment and its parent chapters, relevant metamodel classes and attributes, and enumerations and their literals
Structured output	Constraint identifier, content, type, values, related provisions, and candidate target classes, attributes, or enumerations
Automatic linking	Resolve target names, check types and member ownership, and establish references from constraints to metamodel elements
Exception handling	Have humans correct records with unresolved links or member mismatches, then repeat the linking checks
Subsequent use	Retrieve constraints by element association and task relevance, and connect them to generation and validation through existing mappings and check implementations

All 60 generation runs passed the checks for their declared task profiles. The decision for the full configured rule corpus remained INCOMPLETE, as broader system and deployment properties were outside these component tasks.

Custom checks were written by humans with LLM assistance. The authors consulted the specifications to determine applicable objects, decision conditions, and diagnostic locations, selected implementations, and tested them with satisfying, violating, and boundary cases. Declarative operations address ranges, enumerations, multiplicities, existence, and references. Dedicated checks address relationships across objects. At

runtime, check instances are assembled from rule identifiers and the current object bindings. Validation code is not regenerated for each task.

Post-generation validation first checks the actual ARXML against the AUTOSAR XSD, then resolves local references and executes applicable domain rules. The task acceptance checker reads component names, port sets, interface references, initial values, timeouts, runnable entities, periods, and read/write relationships from the case specification and compares them with the actual artifacts. Agreement between the artifacts and the generation plan is checked separately. For example, a timing event being parseable does not establish that its period is correct. If the task requires

10 ms, the acceptance checker must also verify that the actual value is 0.01 s.

A.3 Core and substitute faults

Controlled repair used reference artifacts that were constructed deterministically from the cases and the XSD and had passed acceptance checks. Five fault slots were assigned to each base case. Fixed core operations were applied to the 85 slots with applicable target objects. Executable substitute operations were specified before the runs for the remaining 15 slots. Repair was limited to two rounds, and every unit was completed within one round in the observed runs.

Substitute operations were needed when the target structure was absent. For example, a case containing only provided ports has no receiver initial value to empty. Similarly, a case without a runnable entity or event cannot receive a fault in the corresponding path. Table A4 lists the substitute operations that were executed. Each row covers three cases.

The reference file sets and their byte contents were restored in 85/85 core units and 15/15 substitute units. The core set used 109 model responses and 661,631 tokens. The substitute set used 21 responses and 54,993 tokens. Some insertion repairs generated content in stages, so a single repair round could involve multiple responses.

A.4 Worked example of periodic component construction and event reference repair

Task and domain basis. ASW-FULL-01 requires the component ASW_Com_Full_Baseline with three provided ports, three required ports, and six interfaces. One runnable entity performs three reads and three writes every 10 ms. The requirement also specifies a receiver initial value of 0 and a timeout of 0.3 s. The metamodel provides internal behaviors, runnable entities, TimingEvents, and their containment and reference relationships. TPS_SWCT_01519 in the ICM provides the basis for checking events associated with a runnable entity declared to execute periodically [28].

Task binding, L1, and assembly. Task analysis binds the internal behavior, runnable entity, and event to IB_Com_Full_Baseline, RE_Com_Full_Baseline, and TE_Com_Full_Baseline, respectively. It also determines the interfaces, variable accesses, and reference targets. L1 assembles a generation schema from the metamodel and these selections to constrain

the required objects and containment hierarchy. The adapter assembles the structured result, inserts the bound fixed values, and serializes the model in the element order required by the XSD. The result is one component ARXML file and six interface ARXML files.

The following excerpt from the component produced in the first experimental generation run shows the event reference and the actual runnable entity declaration. The component and its ancestors are omitted, and a comment identifies the omitted read/write accesses. Only indentation and line breaks around the reference text have been adjusted. The reference itself is complete.

Excerpt A1 Timing event and runnable entity from the experimental generation run

```
<SWC-INTERNAL-BEHAVIOR>
  <SHORT-NAME>IB_Com_Full_Baseline</SHORT-NAME>
  <EVENTS>
    <TIMING-EVENT>
      <SHORT-NAME>TE_Com_Full_Baseline</SHORT-NAME>
      <START-ON-EVENT-REF DEST="RUNNABLE-ENTITY">
        /Components/ASW_Com_Full_Baseline/IB_Com_Full_Bas
        eline/RE_Com_Full_Baseline
      </START-ON-EVENT-REF>
      <PERIOD>0.01</PERIOD>
    </TIMING-EVENT>
  </EVENTS>
  <RUNNABLES>
    <RUNNABLE-ENTITY>
      <SHORT-NAME>RE_Com_Full_Baseline</SHORT-NAME>
      <!-- Three read and three write VARIABLE-ACCESS
      entries omitted -->
    </RUNNABLE-ENTITY>
  </RUNNABLES>
</SWC-INTERNAL-BEHAVIOR>
```

L2 and task acceptance. The experimental generation run passed XSD validation for all seven documents, 22 local-reference checks, and 61 structural obligations defined by the case on its first attempt. The periodic-execution rule was evaluated for one applicable runnable entity and passed. Task acceptance separately compared the actual period of 0.01 s with the required 10 ms and checked the port set and read/write relationships. Event existence, reference resolution, and the correctness of task-specific parameters were therefore checked separately.

Separate controlled fault injection. For the same requirement, a fault was separately injected into an accepted, deterministically constructed reference component by removing the internal-behavior segment from the event reference. The affected content is shown below.

Table A3 Repair results for core faults

Fault	Units	Detected	Reference content restored
Empty initial value	17	17	17
Incorrect initial value	17	17	17
Missing event or internal-behavior path	14	14	14
Missing receiver communication specification	17	17	17
Incorrect XSD element order	20	20	20
Total	85	85	85

Table A4 Substitute operations declared before execution

Cases	Original core operation	Substitute operation	Units restored
MIN-01 to MIN-03	Empty the initial value	Empty the provided-interface reference	3/3
MIN-01 to MIN-03	Set an incorrect initial value	Change the provided-interface reference to a nonexistent path	3/3
MIN-01 to MIN-03	Remove the receiver communication specification	Delete the provided port	3/3
MIN-01 to MIN-03	Remove the event or internal-behavior path	Delete the component name	3/3
MIN-04 to MIN-06	Remove the event or internal-behavior path	Delete the required-interface reference	3/3

Excerpt A2 Event reference after separate controlled fault injection

```
<START-ON-EVENT-REF DEST="RUNNABLE-ENTITY">
  /Components/ASW_Com_Full_Baseline/RE_Com_Full_Baseline
</START-ON-EVENT-REF>
```

The damaged artifact still passed XSD validation, but local-reference resolution could not find the target. The periodic-execution check reported that the runnable entity lacked an associated TimingEvent, and task acceptance also failed. Repair targeted the event's reference text and restored the complete path using the bound target by reinserting IB_Com_Full_Baseline. After serialization, revalidation passed in one round. The file set and byte contents of the component and six interfaces were restored to those of the reference artifacts. This unit demonstrates separate controlled repair. The experimental generation run itself had already passed on its first attempt.

B AUTOSAR-vLLM decoding interventions

B.1 Comparison conditions and execution settings

The experiment used Qwen3.5-9B, vLLM [30], and XGrammar [19] on two RTX 4090 GPUs with tensor parallelism set to 2. The temperature was 0, top_p was 1, and thinking, speculative decoding, and prefix caching were disabled. The maximum output was 16,384 tokens. Each of the 20 cases used three fixed seeds, yielding 60 blocks. Each of the six execution orders for prompt-only constraints U, constrained decoding G, and decoding with auditing A was used in ten blocks.

All three conditions received the same requirement, output instructions, and complete JSON Schema in their prompts. U provided the constraints only through the prompt. G additionally enforced them through the decoding backend. A also recorded interventions at each step. The generation constraints were derived

from fixed case structures and specified fields, arrays, nesting, and values. These inputs were held fixed to examine the effect of enforcing decoding constraints.

B.2 Results by tier and execution cost

Table B1 Structural acceptance by case tier

Case tier	U prompt only	G constrained decoding	A decoding and auditing
Minimal, 6 cases×3 runs	9/18	18/18	18/18
Standard, 7 cases×3 runs	18/21	21/21	21/21
Full, 7 cases×3 runs	18/21	21/21	21/21
Total, 20 cases×3 runs	45/60	60/60	60/60

In the prompt-only condition, three minimal cases represented an array of receiver communication specifications as an object. One standard case and one full case had incorrectly nested access references. All three repetitions failed for each of these five cases. The remaining cases passed structural checks and subsequent artifact acceptance. With decoding constraints enforced, all tiers passed. At the base-case level, five cases improved and fifteen were unchanged.

All 60 audited generations contained steps at which the grammar excluded the highest-scoring unconstrained token. There were 7–10 such steps per generation and 516/54,942 steps in total. All 60 output pairs from the auditing condition and ordinary constrained decoding were byte-identical. Table B2 compares output volume and request time.

Auditing added a mean request time of 2.887 s and a median of 2.634 s. Cumulative request time increased by 27.85%. These times are measured per model request and are separate from the end-to-end AUTOSAR workflow times reported in the main text.

Table B2 Tokens and request time

Metric	U	G	A
Total output tokens	73,644	54,942	54,942
Mean output tokens per request	1,227.4	915.7	915.7
Total request time/s	811.519	621.901	795.092
Mean request time/s	13.525	10.365	13.252

C Railway model generation and repair

C.1 Model sources and checks

The experiment reused the Train Benchmark meta-model and six VIATRA queries [31]. The authors designed three task families for service connections, maintenance preparation, and redundant monitoring and applied them to eight layouts comprising chains, branching and merging tracks, cycles, self-loops, shared switches, separated lines, multiple regions, and branching cycles. The 24 structured TaskSpec records include English task descriptions and were available during development. Task specifications define objects, fixed physical connections, assignable fields, and the service states and monitoring relationships that must be preserved. The system combines these structured requirements with metamodel information to construct task constraints and a structural plan and to bind the current objects and values. Domain checks use the six VIATRA queries from commit 9c76520 of the official Train Benchmark repository. Table C1 summarizes the rule conditions implemented in that version.

After checking field domains and reference scopes, the generated output is assembled into XMI. Validation loads that XMI directly, performs EMF model diagnostics and VIATRA queries, and cross-checks the match sets against independently implemented relational queries. Task acceptance separately checks prescribed states, relationships, and protected content.

C.2 Conditions and supplementary sets

The main experiment used gpt-5.6-luna with low reasoning effort, a temperature of 1.0, a maximum output

Table C1 The six native queries

Query	Property checked in this experiment
PosLength	Segment lengths are positive
SwitchMonitored	Every switch is monitored by a sensor
SwitchSet	For an active route whose entry signal is GO, prescribed switch positions agree with their actual positions
RouteSensor	A route requires the sensors that monitor the switches in its prescribed switch settings
ConnectedSegments	No sensor monitors every segment in a directed walk of six segments; vertices need not be distinct
SemaphoreNeighbor	Adjacent track elements monitored by sensors required by different routes satisfy the shared boundary-semaphore relationship specified by the original query

budget of 128,000 tokens, and three fixed seeds. Initial generation G0 was followed by two branches, self-repair GS and repair with validation feedback GF. In the controlled sets, self-repair S received the public rules, the current model, and the permitted fields, using a self-inspection approach related to Self-Refine [24]. V added validation feedback and an acceptance criterion that prevented regression. F further provided violation locations, local candidates, and dependency information. Each branch allowed at most two rounds, with changes to at most six explicit fields per round. Execution could stop early after rejection or a lack of progress.

Across the 48 fixed-damage inputs, all three repetitions passed for 38 inputs under S and 45 inputs under each of V and F.

Task-obligation faults invert the route activation state required by the task. They comprise 16 cases with an incorrectly deactivated route and 8 with an incorrectly activated route. These inputs pass the native domain queries but violate the task requirements. Native query satisfaction therefore does not establish task satisfaction. The correctness-preservation set checks whether repair retains results that already satisfy the requirements.

Of the 49 failed initial generations in the main experiment, 48 produced models. GF repaired 28 of these models. All 20 remaining models still violated SemaphoreNeighbor. The stopping reasons were 16 proposals rejected for causing regression in existing checks, 3 abstentions, and 1 proposal that removed no violation. The initial-assignment stage rejected 1 further starting point. The current process prevents changes that cause regression but did not find an acceptable repair for every combination of relationships.

Results are paired by condition within each base task. Repeated runs and the corresponding faults are first aggregated within each task, and success-rate differences are then calculated with equal weight for each of the 24 tasks. In Table C3, the difference is the first condition minus the second, expressed in percentage points. Intervals are estimated from 9,999 paired resamples of the base tasks. A nominal error level of 0.05 is allocated across four comparisons, with each comparison reporting a nominal marginal percentile interval of $1 - 0.05/4 = 98.75\%$. The three repetitions remain within their respective tasks, and the number of base tasks is 24.

For example, repair with validation feedback and self-repair succeeded in 51/72 and 29/72 runs, respectively, a difference of 30.56 percentage points. For controlled damage, adding localization information to validation feedback yielded one net additional success. The interval for this difference includes zero and does not establish a clear additional benefit. The statistical documentation in the replication package includes a sensitivity analysis based on resampling the eight railway layouts, complete paired records, and analysis scripts.

The 72 responses for G0 used 371,575 tokens and a total response time of 812.54 s. GS added 81 responses, 464,875 tokens, and 799.19 s. GF added 48 responses, 335,937 tokens, and 635.40 s. The cost of a complete repair branch includes its shared initial-generation cost and the additional cost of that branch. Across all sets, the main railway experiment retained 1,074 responses totaling 6,118,127 tokens and 8,840.70 s.

Table C2 Experimental sets and strict success

Set	Input size	Conditions	Strict success
Generation	Three runs for each of 24 tasks, yielding 72 starting points	G0/GS/GF	23, 29, and 51, each out of 72
Fixed damage	One single-fault and one compound-fault input per task; the compound inputs include 12 with coupled faults and 12 with concurrent faults; each input is run three times	S/V/F	129, 140, and 141, each out of 144
Task-obligation faults	24 inputs, each run three times	S/V/F	67, 69, and 72, each out of 72
Correctness preservation	3 correct inputs, each run three times	S/V/F	9/9 for each condition

Table C3 Success-rate differences and intervals in the main railway experiment

Comparison	Difference in percentage points	98.75% resampling interval in percentage points
Generated models, repair with validation feedback – initial generation	+38.89	[22.22, 56.94]
Generated models, repair with validation feedback – self-repair	+30.56	[15.28, 47.22]
Controlled damage, repair with validation feedback and localization – self-repair	+8.33	[0.00, 19.44]
Controlled damage, repair with validation feedback and localization – repair with validation feedback	+0.69	[−3.47, 5.91]

C.3 Second model and execution cost

The second model was gpt-5.6-terra with low reasoning effort and the fixed seed 104729. It used the same 24 tasks, checks, permitted fields, and repair-round limit. Luna results for the same tasks and seed provide the comparison.

All 18 initial Terra successes were preserved. Each of GS and GF repaired four of the six failed starting points. The two branches both succeeded on 21 tasks and both failed on one task. Each also succeeded on one task on which the other failed. The supplementary runs had a maximum completion budget of 4,096 tokens, and all responses in this batch completed within that limit. Luna and Terra used different output budgets, so the comparison reflects each LLM’s results under its own execution configuration.

Table C4 Results for two LLMs on the same tasks

Condition	Luna strict success	Terra strict success
G0, no repair	6/24 (25.0%)	18/24 (75.0%)
GS, self-repair	8/24 (33.3%)	22/24 (91.7%)
GF, repair with validation feedback	13/24 (54.2%)	22/24 (91.7%)

The 60 retained Terra responses used 338,438 tokens, comprising 301,834 input tokens and 36,604 output tokens.

C.4 Worked example of relational repair for two service routes

BAL-1-01 requires two active routes, r1 and r2, on a fixed chain of six segments. Physical connections, prescribed switch positions, and specified monitoring must be preserved. Sensor n1 monitors w1 and t1, n2 monitors w2 and t2, and n3 monitors t3. Each route must require at least two distinct sensors. The structured task specification fixes the objects and connections. The program uses it to assemble generation constraints that restrict sensor requirements to declared candidates such as n1–n6. The initial model generated with seed 155921 selected r1.requires=[n1,n3] and r2.requires=[n2,n3] and was assembled into the following XMI.

The excerpt preserves the original references. Only line breaks have been adjusted, and surrounding and unrelated nodes have been omitted. XMI identifiers must be interpreted through the identity mapping. Identifiers n12 and n13 correspond to routes r1 and r2; n4, n5, and n6 correspond to domain sensors n1, n2, and n3; n15 and n16 correspond to t2 and t3; and n1, n2, and n3 correspond to semaphores m_boundary, m_in, and m_out. Thus, the original requires="n4 n6" represents [n1,n3] at the domain level. Excerpt C1 Initial XMI and route fields after repair

```
<!-- G0: excerpt of sensors, adjacent segments, and two
routes -->
<sensors xmi:id="n5" id="5" monitors="n15 n21" />
<sensors xmi:id="n6" id="6" monitors="n16" />
<elements xmi:id="n15" id="15" xsi:type="railway:Segment"
  connectsTo="n16" monitoredBy="n5" length="40">
  <semaphores xmi:id="n1" id="1" signal="G0" />
</elements>
<routes xmi:id="n12" id="12" active="true" entry="n2"
  exit="n1" requires="n4 n6">
  <follows xmi:id="n10" id="10" position="DIVERGING"
    target="n20" route="n12" />
</routes>
<routes xmi:id="n13" id="13" active="true" entry="n1"
  exit="n3" requires="n5 n6">
  <follows xmi:id="n11" id="11" position="DIVERGING"
    target="n21" route="n13" />
</routes>
<!-- GF: only r1.requires changes; r2, monitoring, and
connections are preserved -->
<routes xmi:id="n12" id="12" active="true" entry="n2"
  exit="n1" requires="n4 n5">
  <follows xmi:id="n10" id="10" position="DIVERGING"
    target="n20" route="n12" />
</routes>
```

L2 successfully loaded the initial XMI, and EMF diagnostics reported no structural errors. The other five native queries passed. SemaphoreNeighbor returned the following violation, with parameters ordered as

the semaphore, two routes, two sensors, and two track elements.
Excerpt C2 Relational-check diagnostic for the initial model

```
SemaphoreNeighbor: FAIL
tuple = [m_out, r2, r1, n2, n3, t2, t3]
```

In this case, t2→t3 are monitored by n2, required by r2, and n3, required by r1, respectively. This triggers the requirement that the exit of r2 and the entry of r1 refer to the same semaphore. Their actual values are m_out and m_in. Using the diagnostic and local candidates, GF changed only r1.requires to [n1,n2], as shown in the second part of the excerpt. Revalidation recorded empty match sets for all six queries. Both routes remained active, and all seven task obligations and protected-content checks passed. The proposal passed repair acceptance and achieved strict success after one round. L1 restricted the reference candidates, L2 checked relationships across objects, and repair acceptance preserved the service task.

D Private international law decisions

D.1 Cases and comparison conditions

The PIL experiment used 60 synthetic jurisdiction scenarios written in English and constructed by two graduate students in private international law. They independently reviewed the initial reference labels without seeing each other's judgments and then discussed disagreements. The cases were based on the consolidated text of Brussels I bis dated 26 February 2015 [32] and the consolidated text of the EU Trade Mark Regulation dated 1 December 2025 [33]. Decision fields, jurisdiction types, and checks across fields were defined from the meanings of these provisions. A support library organized 58 provision paraphrases. References identify both the legal instrument and the provision, and a fixed lexical retrieval procedure returns at most 12 provisions.

Each condition used gpt-5.6-luna with low reasoning effort, a temperature of 1.0, a maximum output budget of 128,000 tokens, and three fixed seeds, yielding 180 results per condition. Neither the model nor the auditor read the reference answers at runtime. Reference judgments were fixed for the cases before generation, and model outputs were subsequently evaluated using common metrics.

Table D1 Generation conditions

Condition	Input evidence	Structural enforcement	Post-generation processing
Basic prompting (P0)	Case facts and common output requirements	Prompt instructions	Parse and check structure
Retrieval augmentation (P1)	Add retrieved legal evidence	Prompt instructions	Parse and check structure
Structural constraints (P2)	Same as P1	Strict schema	Check structure and release
Full checks (P3)	Same as P1	Strict schema	Audit, one repair, revalidation, and release control

D.2 Auditing and results

The auditor was written by humans with LLM assistance. It checks field types, enumerations, primary and alternative jurisdiction types, defined relationships between conclusions and selected provisions, and whether references belong to the permitted evidence. P3 gives the model the original facts, the same evidence, the original answer, and the diagnostics and permits one revision. If revalidation still fails, a record indicating that the output was withheld is retained.

The primary composite endpoint requires valid structure, compliance with the common delivery rules, compatibility with four requirements derived from the reference judgments, and actual release. These requirements check the selected conclusion, primary jurisdiction type, required evidence, and conditional flag. The evidence requirement is satisfied when the cited evidence has at least one item in common with the nonempty required set. These four field comparisons do not fully assess free-text legal reasoning.

The P3 records that passed delivery checks include four records of withheld outputs. There were 176 actual releases and 143 results meeting the primary endpoint. Repair was triggered in 62 units. Of these, 40 changed from incompatible to compatible with the reference judgments and were released, and 42 repair units ultimately met the primary endpoint. A further 3 changed from compatible to incompatible. Field compatibility and compliance with delivery rules are therefore counted separately.

Table D3 reports differences in the primary composite endpoint for the same cases under the four conditions. The three runs are first aggregated within each case, and differences between conditions are then calculated. Paired resampling uses the 60 cases as its units, with 20,000 resamples and 95% percentile intervals. Each sampled case retains its corresponding results under all conditions.

Full checks produced 42 more results meeting the primary endpoint than structural constraints alone, corresponding to 23.3 percentage points. Adding structural constraints alone yielded a difference of 2.8 percentage points, whose interval includes zero. The replication package retains a sensitivity analysis using 31 design-source groups and adjusted tests for the four comparisons.

Mean citation F1 scores for P0–P3 were 0.449, 0.720, 0.712, and 0.692, respectively. Mean token use per planned unit was 1,177.39, 1,465.77, 1,746.33, and 2,392.31, respectively, with P3 including additional repair. Full checks achieved a higher primary endpoint, while citation overlap and cost followed different patterns.

D.3 Worked example of revising the type of a choice-of-court decision

Facts and structured generation. In case8, both parties are domiciled in Italy and agree on the jurisdiction of Swedish courts, but one party brings proceedings in Italy. P3 run 1 retrieved provisions including Article 25 and 25(1) of Brussels I bis and generated a decision record from the same facts and evidence. Strict output structure constrained the fields and enumerations. The initial JSON passed structural checks and concluded that jurisdiction should be declined in favor of the chosen court.

Observed diagnostics. The auditor identified a mismatch between the conclusion and the primary jurisdiction type. This conclusion requires the primary type agreement, whereas the submitted type *exclusive* also lacked the provision required by its encoded rule. The revision input contained the original facts, the same retrieved evidence, the initial answer, and both diagnostics. At most one attempt was permitted. The

Table D2 Results for four conditions with 180 results per condition

Condition	Valid structure	Valid delivery	Actual release	Reference compatibility	Primary composite endpoint
P0	180 (100.0)	91 (50.6)	180 (100.0)	74 (41.1)	73 (40.6)
P1	178 (98.9)	117 (65.0)	178 (98.9)	102 (56.7)	96 (53.3)
P2	180 (100.0)	118 (65.6)	180 (100.0)	105 (58.3)	101 (56.1)
P3	180 (100.0)	180 (100.0)	176 (97.8)	144 (80.0)	143 (79.4)

Table D3 Success-rate differences and intervals for the PIL primary composite endpoint

Comparison	Difference in percentage points	95% resampling interval in percentage points
Retrieval augmentation — basic prompting	+12.8	[2.2, 23.3]
Structural constraints — retrieval augmentation	+2.8	[−3.3, 9.4]
Full checks — structural constraints	+23.3	[13.3, 33.9]
Full checks — basic prompting	+38.9	[27.8, 50.0]

excerpts below show the relevant fields from the initial and revised answers, preserving the original keys, values, and order of the cited provisions.
Excerpt D1 Relevant fields of the initial decision record

```
{
  "conclusion": "Decline_in_favor_of_chosen_court",
  "forum": "Swedish courts",
  "forum_type": "exclusive",
  "alternative_forum_types": ["agreement"],
  "legal_basis": [
    { "instrument": "BRUSSELS_I_BIS", "provision": "Art.25" },
    { "instrument": "BRUSSELS_I_BIS", "provision": "Art.25(1)" }
  ]
}
```

Excerpt D2 Relevant fields after one revision

```
{
  "conclusion": "Decline_in_favor_of_chosen_court",
  "forum": "Swedish courts",
  "forum_type": "agreement",
  "alternative_forum_types": ["exclusive"],
  "legal_basis": [
    { "instrument": "BRUSSELS_I_BIS", "provision": "Art.25" },
    { "instrument": "BRUSSELS_I_BIS", "provision": "Art.25(1)" }
  ]
}
```

Revalidation and release. The revision exchanged the primary and alternative types while retaining the forum, conclusion, and listed evidence. Both structural checks and auditing passed on revalidation, and the record was released. It changed from incompatible to compatible with the reference judgments and met the primary composite endpoint. In this study, **agreement** represents jurisdiction based on a choice-of-court agreement, while the primary type **exclusive** denotes the predefined category of statutory exclusive jurisdiction. This revision does not deny the possible exclusive effect of a choice-of-court agreement under Article 25(1) [32]. The example illustrates checking and revision of relationships across fields after structural validity has been established.

D.4 Targeted expert review

In addition to automated scoring, experts conducted a targeted review of three specific answers for case19 and case42. In case19, the defendant is domiciled only in China, and the scenario excludes the exceptions listed in Article 6(1). Jurisdiction must therefore be determined under German domestic law. The available facts do not establish whether that law confers jurisdiction [32]. The conditional flag should evaluate the selected conclusion itself, for which the prescribed encoding is false. The model's original true value reflected its interpretation of the latter question.

Its substantive explanation was acceptable, but this judgment was not used to alter the original automated endpoint.

In case42, a French consumer purchases furniture online from a Dutch trader and separately confirms a clause selecting Amsterdam courts. The first P1 answer inferred jurisdiction of the specific courts in Amsterdam directly from domicile in the Netherlands. The experts found that the subsequent text did not correct this error and judged the answer unacceptable, although it passed the automated endpoint. The first P3 answer later made the entire conclusion explicitly conditional on the consumer rules and the requirements for a choice-of-court agreement. The experts judged it conditionally acceptable. A consumer's separate confirmation of the clause does not automatically satisfy Articles 19 and 25(4) [32]. This comparison shows why the structured endpoint and a substantive assessment of the complete answer must be interpreted separately.

These judgments are linked to specific cases and original answers and clarify the scope of the scoring. Complete cases, task configurations, generated artifacts, evaluation results, and analysis entry points are retained for the four experimental tracks in the accompanying replication package.

References

- [1] Kahani, N., Bagherzadeh, M., Cordy, J.R., Dingel, J., Varró, D.: Survey and classification of model transformation tools. *Software & Systems Modeling* **18**(4), 2361–2397 (2019) <https://doi.org/10.1007/s10270-018-0665-6>
- [2] Di Rocco, J., Di Ruscio, D., Di Sipio, C., Nguyen, P.T., Rubel, R.: On the use of large language models in model-driven engineering. *Software and Systems Modeling* **24**(3), 923–948 (2025) <https://doi.org/10.1007/s10270-025-01263-8>
- [3] World Wide Web Consortium: XML Schema Part 1: Structures Second Edition. W3c recommendation, World Wide Web Consortium (October 2004). 28 October 2004. <https://www.w3.org/TR/2004/REC-xmlschema-1-20041028/>
- [4] Poesia, G., Polozov, A., Le, V., Tiwari, A., Soares, G., Meek, C., Gulwani, S.: Synchronesh: Reliable Code Generation from Pre-Trained Language Models. In: *ICLR 2022* (2022). <https://openreview.net/forum?id=KmtVD97J43e>
- [5] Chen, B., Wei, O., Zheng, B., Mussbacher, G.: Accurate and Consistent Graph Model Generation from Text with Large Language Models. In: *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, pp. 130–141 (2025). <https://doi.org/10.1109/MODELS67397.2025.00018>
- [6] Almonte, L., Rengifo, J.I., Guerra, E., Lara, J.: EMF-Kaizen: an intelligent assistant for domain-specific modelling and meta-modelling. *Journal of Object Technology* **25**(3), 99–112 (2026) <https://doi.org/10.5381/jot.2026.25.3.a8> . ECMFA 2026 Journal First
- [7] Object Management Group: Object Constraint Language, Version 2.4. OMG Specification formal/2014-02-03, Object Management Group (February 2014). <https://www.omg.org/spec/OCL/2.4/PDF>
- [8] Object Management Group: XML Metadata Interchange (XMI) Specification, Version 2.5.1. OMG Specification formal/2015-06-07, Object Management Group (June 2015). <https://www.omg.org/spec/XMI/2.5.1/PDF>
- [9] Ma, T., Dai, S., Gao, Y., Xu, F., Fang, L.: A Dual-Stage Framework for Behavior-Enhanced Automated Code Generation in Industrial-Scale Meta-Models. *IEEE Access* **13**, 170943–170959 (2025) <https://doi.org/10.1109/ACCESS.2025.3614174>
- [10] Semeráth, O., Nagy, A.S., Varró, D.: A graph solver for the automated generation of consistent domain-specific models. In: *Proceedings of the 40th International Conference on Software Engineering. ICSE '18*, pp. 969–980. Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3180155.3180186>

- [11] Alaoui Mdaghri, A., Ouederni, M., Chaari, L.: MDE in the Era of Generative AI. In: Ben Hedia, B., Ghazel, M., Monsuez, B. (eds.) *Verification and Evaluation of Computer and Communication Systems. Lecture Notes in Computer Science*, vol. 15466, pp. 113–127. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-85356-2_8
- [12] Petrovic, N., Pan, F., Lebioda, K., Zolfaghari, V., Kirchner, S., Purschke, N., Khan, M.A., Vorobev, V., Knoll, A.: Synergy of Large Language Model and Model Driven Engineering for Automated Development of Centralized Vehicular Systems. *arXiv* (2024). <https://doi.org/10.48550/ARXIV.2404.05508> . <https://arxiv.org/abs/2404.05508>
- [13] El-Gnainy, N., Shanouda, M., Essam, A., Ibrahim, A.A., William, J., Elsharkawy, M., Moustafa, P.A., Al Ansary, M., Mahmoud, H., Moro, A., Salama, C.: AI-Enhanced AUTOSAR Configuration: Efficient Methods for Dataset Generation and Automated Code Production. In: *2024 IEEE 8th Forum on Research and Technologies for Society and Industry Innovation (RTSI)*, pp. 214–219 (2024). <https://doi.org/10.1109/RTSI61910.2024.10761393>
- [14] Samy, A., Moro, A., Taher, M.: Automating AUTOSAR BSW Configuration Generation with Fine-Tuned LLMs and a Compact Intermediate Representation. *Applied Sciences* **16**(17), 8443 (2026) <https://doi.org/10.3390/app16178443>
- [15] Abukhalaf, S., Hamdaqa, M., Khomh, F.: PathOCL: Path-Based Prompt Augmentation for OCL Generation with GPT-4. In: *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering, FORGE '24*, pp. 108–118. Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3650105.3652290>
- [16] Winkler, S., Pilgrim, J.: A survey of traceability in requirements engineering and model-driven development. *Software & Systems Modeling* **9**(4), 529–565 (2010) <https://doi.org/10.1007/s10270-009-0145-0>
- [17] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Proceedings of the 34th International Conference on Neural Information Processing Systems. NIPS '20*, pp. 9459–9474. Curran Associates Inc., Red Hook, NY, USA (2020). https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf
- [18] Geng, S., Josifoski, M., Peyrard, M., West, R.: Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. In: Bouamor, H., Pino, J., Bali, K. (eds.) *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 10932–10952. Association for Computational Linguistics, Singapore (2023). <https://doi.org/10.18653/v1/2023.emnlp-main.674> . <https://aclanthology.org/2023.emnlp-main.674/>
- [19] Dong, Y., Ruan, C.F., Cai, Y., Xu, Z., Zhao, Y., Lai, R., Chen, T.: XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. In: Zaharia, M., Joshi, G., Lin, Y. (eds.) *Proceedings of Machine Learning and Systems*, vol. 7. *MLSys* (2025). https://proceedings.mlsys.org/paper_files/paper/2025/file/5c20ca4b0b20b0bd2f1d839dc605e70f-Paper-Conference.pdf
- [20] Li, L., Dong, Y., Wang, G., Xu, Z., Jiang, A., Chen, T.: XGrammar-2: Dynamic and Efficient Structured Generation Engine for Agentic LLMs. In: *Proceedings of the ACM Conference on AI and Agentic Systems. CAIS '26*, pp. 1009–1022. Association for Computing Machinery, New York, NY, USA (2026). <https://doi.org/10.1145/3786335.3813124>
- [21] Chen, B., López, J.A.H., Babikian, A.A.: Projectional Decoding: Towards Semantic-Aware LLM Generation. In: *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, pp. 1277–1281. Association for Computing Machinery, New York, NY, USA (2026). <https://doi.org/10.1145/3803437.3805571>

- [22] Park, K., Wang, J., Berg-Kirkpatrick, T., Polikarpova, N., D’Antoni, L.: Grammar-Aligned Decoding. In: Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., Zhang, C. (eds.) *Advances in Neural Information Processing Systems*, vol. 37, pp. 24547–24568. Curran Associates, Inc. (2024). <https://doi.org/10.52202/079017-0774> . https://proceedings.neurips.cc/paper_files/paper/2024/file/2bdc2267c3d7d01523e2e17ac0a754f3-Paper-Conference.pdf
- [23] Marchezan, L., Kretschmer, R., Assunção, W.K.G., Reder, A., Egyed, A.: Generating repairs for inconsistent models. *Software and Systems Modeling* **22**(1), 297–329 (2023) <https://doi.org/10.1007/s10270-022-00996-0>
- [24] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K., Welleck, S., Yazdanbakhsh, A., Clark, P.: Self-Refine: Iterative Refinement with Self-Feedback. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) *Advances in Neural Information Processing Systems*, vol. 36, pp. 46534–46594. Curran Associates, Inc. (2023). <https://doi.org/10.52202/075280-2019>
- [25] Ma, L., Liu, S., Li, Y., Xie, X., Bu, L.: Spec-Gen: Automated Generation of Formal Program Specifications via Large Language Models. In: *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering, ICSE ’25*, pp. 16–28. IEEE Press (2025). <https://doi.org/10.1109/ICSE55347.2025.00129>
- [26] Wright, A., Andrews, H., Hutton, B.: JSON Schema Validation: A Vocabulary for Structural Validation of JSON. Internet-Draft draft-bhutton-json-schema-validation-01, Internet Engineering Task Force (June 2022). Work in Progress. <https://datatracker.ietf.org/doc/draft-bhutton-json-schema-validation/01/>
- [27] Tolvanen, J.-P., Kelly, S., Di Rocco, J., Pierantonio, A., Tinella, G.: A framework for evaluating tool support for co-evolution of modeling languages, tools and models. *Software and Systems Modeling* **24**(2), 311–338 (2025) <https://doi.org/10.1007/s10270-024-01218-5>
- [28] AUTOSAR: Software Component Template. Standard 062, AUTOSAR (July 2015). Classic Platform Release 4.2.2; 31 July 2015. https://www.autosar.org/fileadmin/standards/R4.2.2/CP/AUTOSAR_TPS_SoftwareComponentTemplate.pdf
- [29] AUTOSAR: Specification of RTE. Standard 084, AUTOSAR (July 2015). Release 4.2.2; 31 July 2015. https://www.autosar.org/fileadmin/standards/R4.2.2/CP/AUTOSAR_SWS_RTE.pdf
- [30] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J., Zhang, H., Stoica, I.: Efficient Memory Management for Large Language Model Serving with PagedAttention. In: *Proceedings of the 29th Symposium on Operating Systems Principles. SOSP ’23*, pp. 611–626. Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3600006.3613165>
- [31] Szárnyas, G., Izsó, B., Ráth, I., Varró, D.: The Train Benchmark: cross-technology performance evaluation of continuous model queries. *Software & Systems Modeling* **17**(4), 1365–1393 (2018) <https://doi.org/10.1007/s10270-016-0571-8>
- [32] European Parliament and Council of the European Union: Regulation (EU) No 1215/2012 of the European Parliament and of the Council of 12 December 2012 on jurisdiction and the recognition and enforcement of judgments in civil and commercial matters (recast). *Official Journal of the European Union*, L 351, 20 December 2012, pp. 1–32. Consolidated text of 26 February 2015 (2012). <https://eur-lex.europa.eu/eli/reg/2012/1215/2015-02-26/eng>

- [33] European Parliament and Council of the European Union: Regulation (EU) 2017/1001 of the European Parliament and of the Council of 14 June 2017 on the European Union trade mark (codification) (Text with EEA relevance). Official Journal of the European Union, L 154, 16 June 2017, pp. 1–99. Consolidated text of 1 December 2025 (2017). <https://eur-lex.europa.eu/eli/reg/2017/1001/2025-12-01/eng>